applied sciences

MDPI

Article

Towards Software Architecture as an Auditable Practice

Pablo Cruz [1,*], Mauricio Solar [1] and Hernán Astudillo [2]

[1] Departamento de Informática, Universidad Técnica Federico Santa María, Av. España 1680, Valparaíso 2390123, Chile; mauricio.solar@usm.cl
[2] Instituto de Tecnología para la Innovación en Salud y Bienestar (ITiSB), Universidad Andrés Bello, Calle 1 Oriente 1180, Viña del Mar 2530959, Chile; hernan.astudillo@unab.cl
* Correspondence: pablo.cruz@usm.cl

Abstract

A system's architectural design plays a vital role in its quality, since quality attributes are system-wide and impacted by the architecture. The evaluation of a software architecture aims to assess its suitability for the purpose of the system, making this practice a core component in the software quality assessment toolkit. Experience shows that evaluating an architecture is not straightforward, and key practical guidance for evaluation progression is a major challenge in real-world cases. This article presents a software architecture evaluation guiding and progression assessment mechanism based on the identification of five essential elements: Architecture Description, Quality Attributes, Business Goals, Architecture Decisions, and Evaluation Adoption. To describe them, this work proposes an extension of the SEMAT Kernel, where each evaluation "essential" is represented as an Alpha with a set of States that depict the (healthy) progression of architecture evaluations. The practicality and usefulness of the approach is assessed with two case studies derived from two previously executed real-world architecture evaluations. The results suggest that when using this conceptualization and description to guide and assess architecture reviews in legacy systems under classic development and maintenance approaches, architects and stakeholders can better understand how to guide and audit the progression of an architecture review; have a ground for reporting results; and regain focus on the evaluation in some scenarios. A key directly derived future research direction is to evaluate the suitability of the proposal in agile-based development contexts. The authors expect that a wider use of this principled definition of the key elements for software architecture evaluations will provide practical and concrete guidance to evaluators, allowing stakeholders to assess specific evaluation efforts, and to eventually improve teaching and learning of the evaluation practice.

Keywords: software architecture evaluation; software engineering

Check for updates

Academic Editor: Jose María Alvarez Rodríguez

Received: 29 December 2025
Revised: 7 March 2026
Accepted: 9 March 2026
Published: 20 March 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

A software architecture is decisive in achieving software quality [1–3]. Quality goals for a software system are the key concerns that shape both the architecture design process and the architecture itself [4]. At the same time, when a software architecture is designed, the constraints of many quality attributes are determined by the chosen architecture [5]. Although software architecture is not the sole determinant of software quality, its criticality comes from the fact that any attempt to improve quality after a poorly designed architecture is often futile [6]. An inadequate software architecture design will also hinder the adoption of modern software engineering approaches such as continuous software engineering due

to the lack of support for related practices, notably test and integration automation [7], rapid delivery [8], and continuous software evolution [9].

The evaluation of a software architecture plays a vital role in software quality [10] at design time [11]. An architecture evaluation is used to identify architectural risks [6] and to determine if quality requirements were addressed in the architecture design [12]. Experience shows that evaluating software architectures is an integral part of software architecture design [13,14], although it is commonly carried out in an informal way [15]. The importance of evaluating a software architecture has been stressed even in agile development [16], where the quick delivery of software features often competes with long-term quality goals [17].

The authors of this work have been working in the software architecture evaluation domain for several years already, performing architecture evaluations for many types of software systems and in various organizational contexts; some of these experiences have been published in peer-reviewed venues [18–21]. By embracing a reflection-in-action approach [22], the authors have analyzed the key elements that architecture evaluations work with, and how these elements progress through an evaluation endeavor.

The SEMAT Kernel [23,24] provides both a framework for describing software practices in terms of the essential elements that practitioners must work with (called "Alphas") and a notation for representing software practices [25]. The SEMAT Kernel is defined as an actionable, extensible, and practical [23] thinking framework for the description of software engineering practices. The extensibility of the Kernel means that in addition to the standardized "Alphas" or "essentials" of software engineering, more "Alphas" can be added to the Kernel to define more concrete software practices.

This paper proposes a mechanism to guide and assess the progression of the evaluation of software architectures. This study aims to answer the following research questions:

- What are the essential elements that need to be considered when evaluating a software architecture?
- What are the discernible progression states of these essential elements that can be used to assess the progression of the evaluation of software architectures?

The mechanism proposed in this paper is based on the identification and representation of five essential elements. Each essential element is operationalized as a SEMAT Alpha with States and checklists. The five Alphas proposed in this work are: Quality Attributes, Architecture Description, Architecture Decisions, Business Goals, and Evaluation Adoption. The use of the SEMAT Kernel as a thinking and modeling framework to represent these essential elements as "Alphas" is a process that is colloquially known as "essentializing a practice" [25]. The use of the SEMAT Kernel as a thinking and modeling framework enforces the practical and actionable aspects of this representation, making it usable for guiding and assessing the progression of an architecture evaluation.

The authors have experience using the SEMAT Kernel to represent and guide software engineering practices. In [26], the authors explored the suitability of the SEMAT Kernel notation to express the architecture evaluation practice, and a conceptual validation was provided. The emphasis shifted from testing a notation to express a software engineering practice to modeling the software architecture evaluation practice. Its use in actual architecture evaluations allowed the authors to find several key issues, mostly related to the progression of the Alphas; for example, the "Working well" State in the Alpha "Evaluation Adoption" [26] was not consistent with how activities are done in an architecture review; thus, a better articulation for progression from State 2 ("Method integrated and in use") to State 3 ("Working well") had to be reworked.

This work contributes to the software architecture practitioner and research domains by identifying the essential elements (i.e., "Alphas") of software architecture evaluations,

each characterized by a set of progression steps (i.e., "States"). In two real-world cases, this definition was used to guide and audit the progression of software architecture evaluations. This work also contributes with a graphical SEMAT-based representation of this mechanism as "Alpha State Cards", which has been made available on GitHub (https://github.com/pcruzn/arch-eval-semat, accessed on 8 March 2026). Other preliminary results have shown an expedient use of the proposal for organizing software architecture evaluation teaching material, as well as a valuable aid in learning the architecture evaluation practice.

The remainder of this paper is structured as follows: Section 2 surveys the background and related work, including the SEMAT Kernel; Section 3 presents the research strategy; Section 4 introduces the proposed "Alphas" for a software architecture evaluation, their progression paths and relationships, and how to use them to guide and assess a software architecture evaluation; Section 5 describes two actual architecture evaluation efforts that have been guided using this proposal; Section 6 discusses the case studies and key insights; Section 7 addresses threats to validity; and Section 8 summarizes and concludes.

2. Background and Related Work

2.1. Software Architecture Evaluation

There is a widely agreed assumption that the quality of a software system is greatly determined by the high-level design embodied in its software architecture [1,2,4,5]. Therefore, evaluating a software architecture is a key practice in achieving software quality.

In general, a software architecture evaluation aims to identify risks in the proposed architecture [6,12,27], as well as to determine whether quality concerns and quality requirements have been addressed at the architecture level [1,4,15].

Rather than answering "yes" or "no" to a general question about the suitability of the architecture for the purpose of the system, an architecture evaluation typically reports risks considering the purpose of the system [6] and provides concrete evidence on the degree to which the system meets its quality criteria [28].

Experience shows that architecture evaluation is a key practice when designing software architectures in the industry [13], although it is frequently carried out informally [15].

Two general approaches are used to evaluate design at the architecture level: questioning, and measuring [12]. When using a *questioning* approach, the evaluation is based on a set of qualitative questions that are applied to a software architecture to assess how well the design supports the question. Examples of the questioning approach are the use of scenarios, questionnaires, and checklists [12]. On the other hand, the *measuring* approach includes the use of metrics, simulations, prototypes, and experiences [12].

There is an evident tradeoff between the two approaches: applicability. Questioning techniques are applicable to assess the architecture in light of any quality attribute [12]. Measuring techniques bring more specific answers and, therefore, they are applicable to a more specific set of qualities [12].

An architecture evaluation can be completed in a systematic and repeatable way by following an evaluation method; depending on the method's intention, it adopts one or another of the aforementioned approaches and techniques. For example, the Architecture Tradeoff Analysis Method (ATAM) [6] uses the scenario technique to operationalize the quality attributes that will be *questioned* in the evaluation. Specific measure-based evaluations can be designed; for example, ref. [29] evaluates the *resilience* of self-adaptive systems, using a simulation-based technique.

Although a software architecture can eventually be evaluated in an ad hoc way, the use of a method supports repeatable analysis [30]. As of this writing, there are several methods for evaluating an architecture, each embracing a specific approach. The Software Architecture Analysis Method (SAAM) [28] uses scenarios to characterize the circumstances

in which the software system will be used to assess how well the proposed software architecture will satisfy the specifications expressed in the scenarios. The method works by *questioning* how well the architecture supports elicited scenarios, distinguishing between direct scenarios (i.e., those that are directly supported by the architecture) and indirect scenarios (i.e., those that require some changes in the architecture to support them).

Another scenario-based method is the Architecture Tradeoff Analysis Method (ATAM); it goes further by not only using scenarios to *question* the architecture in light of quality criteria, but also analyzing how these quality goals interact (or "trade off" between them).

The Scenario-based Re-engineering Method (SBAR) [31] presents a re-engineering approach using a scenario-based analysis to assess the current architecture and decide if a transformation is required; unlike the previous methods, SBAR highlights in its openness to embrace other approaches for architecture assessments [31].

The Decision-Centric Architecture Reviews Method (DCAR) [32] takes a different approach: it follows a decision-based architecture evaluation where the software architecture being evaluated is characterized as a set of design decisions that are assessed against potential cases in which the decision would be challenged [32].

The Pattern-Based Architecture Reviews Method (PBAR) [33] emerges as a lightweight pattern-based architecture assessment method; it proposes to examine the architecture to identify the architecture patterns used and to determine if these patterns align with the quality goals of the software. Its authors argue that by focusing on identification of established patterns, the analysis is suitable for small and production-focused teams [33] where architecture documentation is expected to be sparse.

Running an architecture evaluation is deemed a challenging effort. Challenges appear in the evaluation itself because of the gap between understanding an evaluation method and bringing it into practice. Some authors [34] argue that ATAM has too steep of a learning curve for practical use. Successful practitioner-oriented books on software architecture (e.g., [35]) highlight the lack of real-world focus on some of the methods for analyzing architectures, sometimes making use of an evaluation method too dependent on personal experience [36].

Several efforts have been made to overcome these challenges. For example, experience reports (e.g., [34,37–40]) are a well-known way to share experiences about architecture evaluations, the challenges the reviewers faced, and how they dealt with those challenges. Some authors [41] have researched the factors that influence the use and success of an architecture evaluation practice. Knodel and Naab's book [42] is an interesting development of the common phases for running software architecture evaluations; it condenses the results from many experiences that they were involved with. There are also "standard" sources, such as the "Software Architecture Review and Assessment (SARA) Report" by Obbink et al. [43] and the ISO/IEC/IEEE 42030:2019 "Software, systems and enterprise—Architecture evaluation framework" [44,45].

None of these efforts has presented a more actionable representation of architecture evaluations for practical use. An actionable representation should be concrete enough so that practitioners can understand the progression of architecture reviews and act upon the determined progression state.

2.2. The SEMAT Kernel and the Essence Standard

The SEMAT Kernel is one of the most prominent results derived from the "call for action" that Jacobson, Meyer and Soley initiated in 2009 [46], aiming at refounding the software engineering discipline based on a sound theoretical framework. The SEMAT Kernel aims at providing this theoretical framework by standardizing the essential elements that any software engineering endeavor should do and progress [47].

The SEMAT Kernel along its description language (both in graphical and textual forms) eventually became standardized in the OMG (Object Management Group) Essence Standard [24,25]. The description of a practice such as the evaluation of software architectures using the SEMAT Kernel is colloquially known as "essentializing" the practice [25], which should be understood as using a common standardized way of expressing the practice rather that trying to change its nature.

The Kernel tries to achieve a good balance between being too specific, and therefore losing general applicability, and being too general, and therefore ending up in describing any kind of endeavor without considering software engineering [24]. To overcome this challenge, the Kernel is designed to be extensible [48], meaning that more specific practices can be described (or essentialized) using the elements that the Kernel provides.

In its base and standardized form, the SEMAT Kernel [24] defines seven "Alphas" that capture the key concepts in software engineering, serving as a common ground for expressing methods and practices. Each "Alpha" defines a series of States that focus on the assessment of the progression and health of a software engineering endeavor. The seven "Alphas" are Opportunity (dealing with the justification for the software system to be developed or changed), Stakeholders (the people, groups or organizations that affect or are affected by the software system product and its engineering), Requirements (the expectations that stakeholders have with the system), Software System (the software system as a product considering software, hardware and data, to provide its primary value), Work (the mental or physical effort to achieve a result), Team (the people involved in the development, maintenance, delivery or support of the software system), and Way of Working (the set of practices and tools that are used by the team to guide and support their work).

In addition, the Kernel organizes the elements in three areas of concern: Customer (dealing with the use and exploitation of the software system, represented with color green, and encompassing the Alphas "Stakeholders" and "Opportunity"), Solution (containing everything related to the specification and development of the software system, represented with color yellow, and encompassing Alphas "Requirements" and "Software Ssystem"), and Endeavor (related to the teams and the way the teams approach their work, represented with color blue, and encompassing Alphas "Work", "Team", and "Way of Working").

The Kernel also defines the work that practitioners do in a software engineering endeavor. For the Kernel, this work is expressed as "Activities". The Kernel avoids the definition of specific "Activities" [25]. Rather, it defines a set of "Activity Spaces" that act as placeholders to group specific "Activities" that practitioners are expected to fill. The current "Activity Spaces" are [24]: Customer Area of Concern: Explore Possibilities, Understand Stakeholders Needs, Ensure Stakeholders Satisfaction, Use the System; Solution Area of Concern: Understand the Requirements, Shape the System, Implement the System, Test the System, Deploy the System, Operate the System; Endeavor Area of Concern: Prepare to do the Work, Coordinate Activity, Support the Team, Track Progress, and Stop the Work.

As noted before, the Kernel is extensible and has been previously extended to describe the design of microservice architectures [49], to represent the test-driven development practice [50], and to guide and assess software architecture evaluations [26], to mention a few cases.

3. Research Method

The research method used in this work consists of two strategies: the use of a reflection-in-action approach with literature research, and the study of real world cases (see Figure 1).

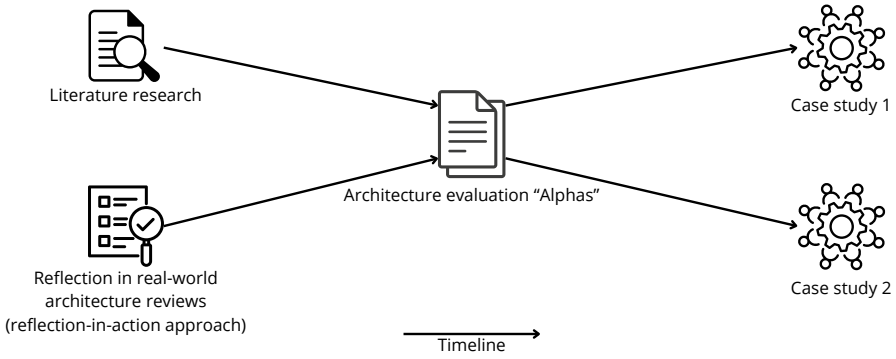


Figure 1. The research strategies used in the research method.

In the first strategy, the architecture evaluation leader used the reflection-in-action [22] approach to obtain reflectively involved in the practical work of several architecture evaluations. The reflection-in-action approach is frequently used in software engineering research [47,51]. In practical terms, a reflection-in-action approach means a continuous research-oriented review of the architecture evaluation practice-related activities. Literature research is used to support a solid ground for the domain of such evaluations. In a reflection-in-action approach, the interest is in both etic and emic issues [52]; that is, those key insights that serve as a conceptual structure for the study of the architecture evaluation phenomenon and those key insights that appear and emerge when running an architecture evaluation and reflecting in, and on the action, respectively. Etic and emic issues are deeper aspects that are of interest to researchers [52]. Retrospectives were used for this purpose, serving as more formal instances where the team reflected in, and on the architecture evaluation activities and their results. This first strategy was used to deeply study the architecture evaluation phenomenon in its real-world context and to define the essentials of such evaluations and their progression paths towards the completion of the evaluation effort.

The second strategy is the use of case studies. Case studies are empirical enquiries [53] that are commonly used in software engineering [53] when the phenomenon of interest is strongly affected (although not necessarily determined) by the context [54], making other empirical methods hard to apply due to lack of experimental control [53]. Caution must be exerted for unnecessary research intrusion to avoid the case study from degrading into an artificial artifact [55]. Case studies typically make use of an observational approach that aims at collecting insights as the case goes through its execution [56]. The study of cases focuses on the interpretative understanding [54] of a real-world phenomenon by engaging in a "dialogue" with those who are part of the real-world context [54]. In this work, each case corresponds to an evaluation of a software architecture in a real-world context with a real software product.

Case studies can take an analytical generalization (rather than statistical) approach [57], as well as a naturalistic approach [54,58]. In the first approach, the case is predefined with an analytical generalization focus [57], trying to identify general and universal patterns [54]. In contrast, the naturalistic approach [54] is oriented toward bringing out the so-called emic issues, that is, the key insights that emerge from a case in a real-world context [52]. Therefore, a key aspect of a case study that takes the naturalistic approach is that the context in which the case is observed and studied is real world, that is, the ordinary setting and natural habitat of the case [54]. Case studies often take a narrative approach [59] to report the case. A narrative approach is also common in software engineering research, especially in practitioner-oriented research reporting [60].

A naturalistic approach was adopted because one of the authors had the opportunity to run two software architecture evaluations in two varied real-world contexts. The first case is the evaluation of a human resources software system in a military institution. The second case is the evaluation of a health record management system in a public cancer treatment-oriented hospital. For reporting the cases in this article, a narrative form was adopted with an emphasis on how the cases proceeded. The key insights that appeared in these cases are also discussed. The case study designs followed logistic aspects such as the planning of evaluation meetings, the use of meeting rooms with specific support tools such as modeler software and whiteboards, as well as meeting several "entrance requirements" because both institutions where the cases took place had privacy-sensitive data. In both cases, short retrospectives were used to iteratively evaluate the use of this proposal in the architecture evaluation. The many emic issues that appeared outside the retrospectives were also written down. The reporting of the two cases in this paper is followed by a discussion where key insights are highlighted, both observed and reported by the evaluation participants.

This proposal has been used in several architecture evaluations. However, two of such evaluation experiences were chosen to be reported as case studies in this paper. The rationale behind the selection of these two experiences as case studies in this article is summarized as follows:

- Criticality of the system for the organization: the authors believe that if a system is critical for the organization, the experience of evaluating their architectures would be closer to real-world situations. In the experience of the authors, stakeholders take a more serious stance in architecture evaluation if the system is critical to the organization. Both systems evaluated and reported in this article are considered critical to their organizations and therefore were chosen to be presented in the article.
- Size of the software system: the size of a software system is correlated with the size and complexity of the architecture. A bigger architecture gives more space for evaluation activities, and therefore, the authors expected more issues to be observed and learned from evaluating bigger architectures. Both architecture evaluations presented in the article are considered big in terms of subsystems (both systems exhibiting more than 15 subsystems; most of these subsystems are software systems in themselves).
- Diversity of stakeholders: evaluating a software architecture is a stakeholder-intensive endeavor. The authors believe that a greater diversity of stakeholders in an evaluation is closer to the context where the proposal is expected to be used (i.e., with many people not being experts in architecture evaluation). Both experiences presented in the article exhibit this characteristic: a very diverse set of stakeholders.

In all cases, all participants were informed that they will be working with a research-oriented approach, and thus some part of the experience will eventually be published. The stakeholders agreed in all cases to this, always asking not to disclose critical or structural information. Previously published experiences were presented to stakeholders. In this way, stakeholders understood that such papers only report the experience and not any related software assets (e.g., architecture descriptions), unless explicitly approved by the involved parties. In addition, the evaluation participants were always instructed to avoid disclosing sensitive data (e.g., patient information), as concrete instances of such assets are typically not of much interest in the architecture evaluation.

4. The Essentials of Software Architecture Evaluations

4.1. SEMAT Kernel Definitions

An Alpha (acronym: Abstract-Level Progress Health Attribute) is an essential element that is considered relevant to the assessment of the progress and health of a software engineering effort [24]. An Alpha represents and holds the discernible States that can be observed in the progression of a software engineering effort. In other words, the Alphas express the subjects whose evolution in such efforts the practitioners want to understand, monitor, direct, and control [24].

A State in an Alpha is a discernible and uniquely identified characterization of the status of an Alpha in a specific moment in a software engineering endeavor [24]. An Alpha comprises a set of one or more States that represent its evolution from its creation to its termination. States are characterized in terms of a set of checklist items that must be achieved to indicate that the state has been reached. The graphical notation provided by the SEMAT Kernel represents an Alpha and its comprised States by means of an "Alpha Card" (see Figure 2), which contains the names of the States, the name of the Alpha, and a brief text that describes the essential that is represented by the Alpha. The use of a card to represent Alphas is a design decision that allows practitioners and researchers to "touch" the Kernel [23] and therefore use it in a practical, concrete form. The color expresses the "Area of Concern:" (blue for "Endeavor"; yellow for "Solution"; green for "Customer").

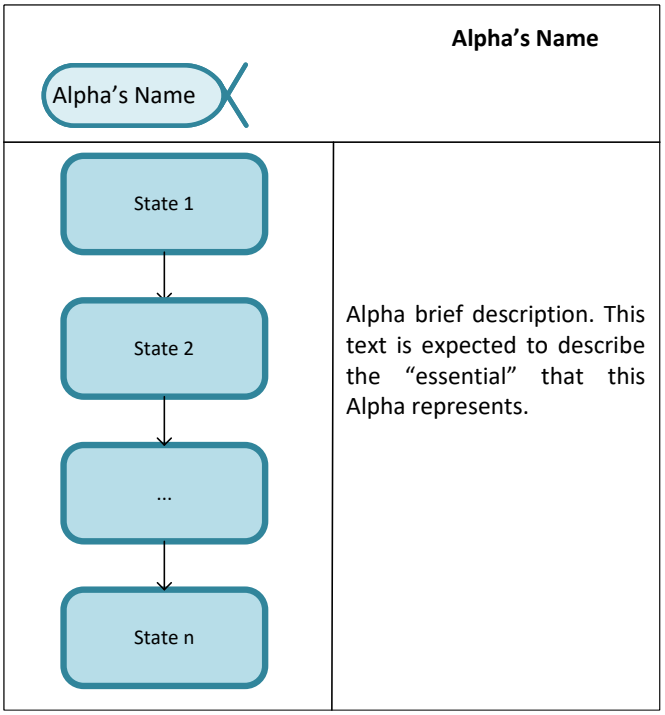


Figure 2. The SEMAT Kernel graphical notation for an Alpha and its States (the arrows indicate the progression direction of the Alpha).

There are two types of associations between Alphas in the SEMAT Kernel. The first one is the containment association, which expresses that an Alpha is subordinated to a superordinate [24]. This association indicates only that the "sub-alpha" contributes to the progression of the "super-alpha" and there is no explicit relationship between both sets of States. The second kind of association between Alphas is the general Alpha Association. Unlike the first one whose semantics are defined by the Kernel, this second kind of association expresses a relationship between two or more Alphas with a semantic

that is indicated by a software practice [24]. For example, in this work, the alpha "Quality Attributes" is considered a subordinate to the alpha "Requirements" and, at the same time, the Alpha "Quality Attributes" is influenced by (relationship) Alpha "Architecture Decisions." The semantics for this second association is defined by software architecture and software architecture evaluation practices.

A practice is defined as a repeatable approach to doing something with a specific objective in mind [24]. A practice is expressed in terms of Alphas and the relationships between the Alphas express the mechanisms by which these Alphas interact according to the practice. Modeling a practice using the SEMAT Kernel means identifying the essential elements of a practice and expressing them as Alphas and their relationships. This process is colloquially known as "essentializing" a practice [25]. The five essentials of the software architecture evaluation practice identified in this work are expressed as the following Alphas: Business Goals, Architecture Description, Architecture Decisions, Quality Attributes, and Evaluation Adoption.

The SEMAT Kernel also provides a graphical notation for expressing the criteria that each State of an Alpha requires to mark the State as reached. The criteria are expressed as a checklist on an "Alpha State Card" [23,24]. Figure 3 shows a generic Alpha State Card with the checklist composed of checkpoints [24]. Each checkpoint must be marked as achieved to say that the State represented in the card has been reached. The same figure shows that states can be numbered: in this case, State *i* from *k* States.

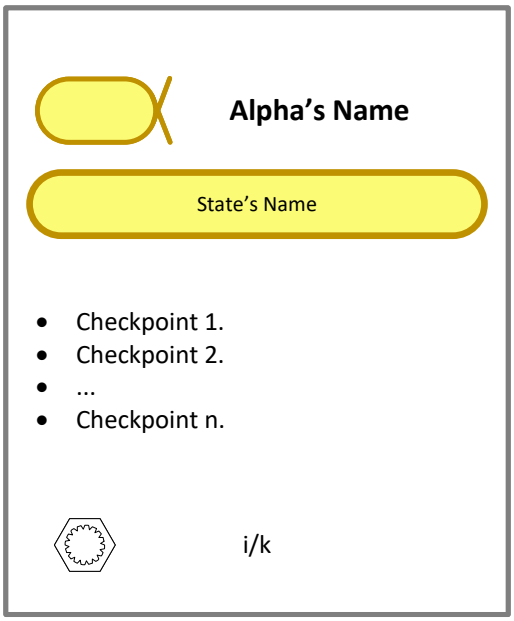


Figure 3. A generic Alpha State Card for a specific State.

4.2. An Argument in Favor of the SEMAT Kernel

A natural question to answer here is why the SEMAT approach was adopted in this work. The argument in favor of SEMAT is supported by the following aspects:

- A software architecture evaluation is an endeavor, that is, a conscious and concerted effort devoted to the assessment of a software architecture fitness for the purpose the system was conceived for. This is the approach that was embraced by the SEMAT Kernel [24,25], that is, the representation of software engineering practices that run as endeavors.
- The SEMAT Kernel defines a common ground for expressing methods and practices in software engineering endeavors [24,25].

- The SEMAT Kernel defines both a graphical and a textual language to describe practices and methods in terms of the essential elements that can be identified after a careful study of a software practice [24,25].

The SEMAT Kernel focuses on software engineering endeavors. A software architecture evaluation is an endeavor. Regardless of the moment the evaluation is run, it is a temporary effort. Therefore, the SEMAT approach is a natural match for the purpose of representing software architecture evaluations. Frameworks such as the successful Capability Maturity Model Integration (CMMI) [61,62] focus on evaluating and improving the organizational-level (rather than the endeavor-level) software process. Moreover, while the CMMI provides a collection of practices for this goal, it does not provide a *"kernel"* of essential elements from which new practices and methods could be expressed. To avoid confusion, it must be noted that the SEMAT Kernel and the CMMI are not incompatible; indeed, practices from the CMMI can be expressed in the SEMAT standardized language. The standard ISO/IEC/IEEE 42030:2019 "Software, systems and enterprise: Architecture evaluation framework" [44] provides a generic conceptual framework for the evaluation of software, systems, and enterprise architecture. The standard is seen as a complement for the standard ISO/IEC/IEEE 42020:2019 "Software, systems and enterprise: Architecture processes" [45], which define a generic software, systems and enterprise architecture evaluation process (among others) by providing generic recommendations for activities and tasks, as well as related work products. While both standards provide very valuable conceptualizations and recommendations for a process for architecture assessment, they do not provide a foundation of the mechanisms by which the concepts work, and they also do not provide a framework from which expected progression on the activities could be drawn upon.

The second aspect from the SEMAT approach is the availability of a "Kernel" of widely-agreed essential elements that software engineers need to progress in any software engineering endeavor. At the same time, this "Kernel" emerges as a strong common ground for expressing methods and practices [24,25]. As mentioned in Section 2, these essential elements are called *"Alphas"* and each one of these elements has a progression path that is used to assess where software engineers are at in an architecture evaluation, as well as to understand the next steps towards a healthy practice run. The SEMAT Kernel is extensible, which means that a new practice (or set of practices, i.e., a method [25]) can be represented using the Kernel basis as a foundation for the representation. This is also a key characteristic that makes the SEMAT Kernel very appropriate for the purpose of this work. In addition, the SEMAT Kernel became an OMG Standard [24], and as such, it provides a standardized way to model software practices.

Finally, the third important aspect is that the SEMAT Kernel offers both a graphical and a textual notation. This allows us to focus on the efforts in the identification of the essential elements for the representation of a software architecture evaluation rather than in designing a new language for these purposes.

4.3. Identifying the Essential Elements of a Software Architecture Evaluation

This work argues that the software architecture evaluation practice characterized as an endeavor deals with the following "essentials" or key elements: Quality Attributes, Architecture Description, Architecture Decisions, Business Goals, and Evaluation Adoption. By adopting the SEMAT Kernel as a modeling framework, this work commits to the following aspects:

- The essentials are represented as Alphas, that is, key elements with which software practitioners work in a software architecture evaluation effort.

- These Alphas comprise a set of predefined States that represent the progression path that a healthy software architecture evaluation should follow.
- The graphical notation for the Alphas and their States is adopted. Figure 4 shows the Alphas proposed in this work using the SEMAT's graphical notation. The black diamond indicates the subordination of the proposed Alphas to the existing Alphas in the SEMAT Kernel.

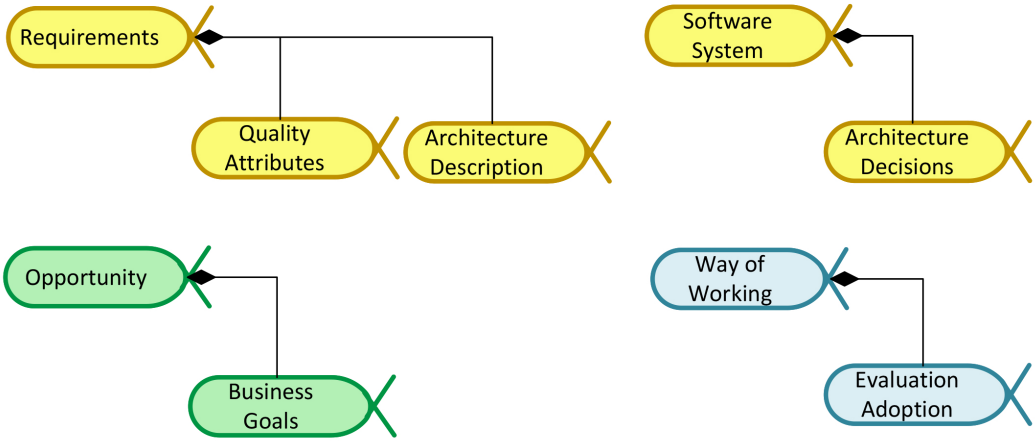


Figure 4. The essentials (Alphas) of a software architecture evaluation represented in the SEMAT Kernel graphical notation (the diamond indicates the containment relationship and the color expresses the "Area of Concern").

Identifying the Alphas of the software architecture evaluation practice is a key aspect of this research. The Alphas that are identified in this work constitute an extension to the original SEMAT Kernel. Required by the SEMAT Kernel, each Alpha comprises a set of discernible States that represent the progression from starting the work with the Alpha to finishing the work with the Alpha. According to the SEMAT Kernel, to achieve a State, the checklist of that State must be fulfilled [23]. The authors believe that when time and cost constraints arise, a more relaxed version of this rule could eventually be used: that is, allowing a State achievement by fulfilling a subset of the checklist (i.e., core plus desired items to be fulfilled), although more study is required to define that subset. The authors remark that this study considers the original SEMAT rule, that is, for a State to be achieved, the checklist must be completely fulfilled. This is the rule that the authors have been enforcing when using this proposal, including the real-world cases presented.

When modeling a practice, there are elements that are inevitably excluded from the set of *essential elements*. For example, scenarios are an important way to articulate quality attribute concerns. However, they are just one of the alternatives. For example, the goal-oriented software requirement engineering [63] provides another approach for dealing with quality attribute concerns. Therefore, "Quality Attributes" (rather than scenarios) is proposed as one of the essential elements (i.e., "Quality Attributes" is a new Alpha) that need to progress in evaluating a software architecture.

In the following, the definitions for each of the new identified Alphas and their States are provided. In addition, Appendix B presents the checklists for each State in each Alpha. These checklists allow for more actionable guiding and progression assessment in an architecture evaluation effort. The checklists are also available as Alpha State Cards on https://github.com/pcruzn/arch-eval-semat (accessed on 8 March 2026).

Figure 5 presents each Alpha with their own States in hierarchical form.

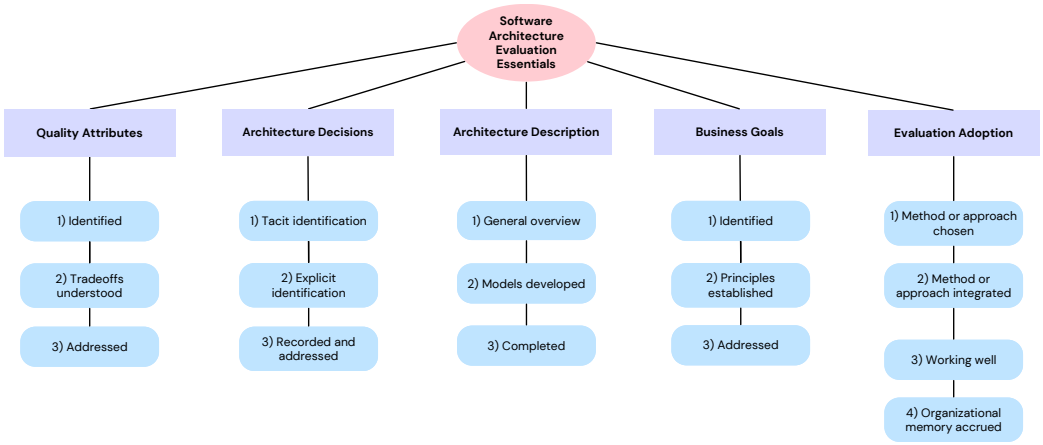


Figure 5. The essentials ("Alphas") of a software architecture evaluation with their States.

Hereafter, a "title case" is used when referring to "Alphas", and a "sentence case" (or simply lower case) is used when referring to a concept with the same name as the "Alpha." For example, "Business Goals" refers to Alpha "Business Goals" and "business goals" refers to the goals that someone (or a company) has.

4.4. Alpha: Quality Attributes

The Alpha "Quality Attributes" represents the properties of a software system that are used to define quality in a specific context. Quality attributes are system-wide [64], and therefore system-level design (i.e., the software architecture) has a profound impact on them. In fact, a great part of the architecting of a system is committed to making decisions to satisfy quality attribute requirements [2,65]. Quality models such as the ISO/IEC 25010:2023 [66] comprise a balanced set of quality attributes. Because quality attributes are typically hard to characterize, such quality models often provide more than one level for quality characteristics (e.g., characteristics and sub-characteristics in [66]). In addition, quality models are often recommended to avoid over representing a few quality attributes [5] and balancing stakeholder needs [67].

In an architecture evaluation, the people involved need to understand and characterize quality attributes because a software architecture, i.e., the set of architecture decisions, greatly affects these attributes. Some methods such as the Architecture Tradeoff Analysis Method (ATAM) [6] explicitly mention quality attributes. In the authors' experience [19], quality attributes always come into play when evaluating a software architecture even if the method does not explicitly state working with them (for example, the Decision-Centric Architecture Reviews [32] method does not explicitly progress quality attributes understanding). This approach is also recognized in [68], where the authors argue that a reasoning framework requires working with quality attributes to determine unmet requirements and eventual changes in the architecture.

Characterizing software quality attributes in a way that they can be evaluated is a challenge [65]. Quality attributes tend to be very general in application, and thus, a characterization is needed for practical use in architecting a software system, as well as evaluating its architecture. Approaches such as utility trees alleviate this problem by leading people to reason about quality attribute's concerns and concrete scenarios [17]. However, scenarios are just one more approach for characterizing quality attributes. For example, goal-oriented software requirement engineering [63] provides another approach to deal with quality attribute concerns.

The proposed Alpha "Quality Attributes" comprises three States (see Figure 6): "Identified", "Tradeoffs understood", and "Addressed".

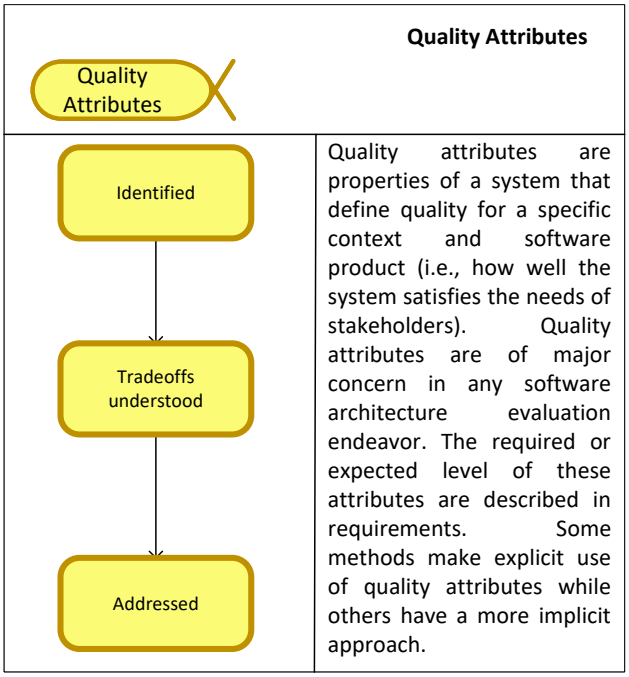


Figure 6. The States of Alpha "Quality Attributes".

The States are described as follows:

1. **Identified:** Reaching this state means that quality attributes are explicitly identified and recorded. In practice, this might imply that the organization is signing off the acceptance and use of a specific quality model. Quality attributes are characterized as requirements, scenarios, goals, or other approaches.
2. **Tradeoffs understood:** Reaching this state means that quality attributes have been prioritized (i.e., their relative importance in regard to the software system whose architecture is being evaluated). This prioritization has been used to understand tradeoffs between quality attributes.
3. **Addressed:** At this state, prioritized quality attributes have been considered in the architecture evaluation. Tradeoffs have been explained and recorded.

4.5. Alpha: Architecture Decisions

The Alpha "Architecture Decisions" represents the design determinations that a software architect has made about the structure of a software system to enable software requirements to be met when treated as first-class entities [69].

Software architecture has been conceptualized using many approaches. For example, Perry and Wolf [70] conceptualize software architecture as a set of architectural elements with a specific form (structure) and a rationale explaining the placement and connection of concrete architectural elements. Perhaps the most practical conceptualization for software architecture is the one that conceives a software architecture as a collection of design decisions [1,71]. This latter focus has motivated a wide variety of architecture activities such as knowledge management and sharing [72]. In this sense, a key aspect of architectural decisions is that they affect quality attributes (positive or negative), although this interaction is sometimes hidden by architectural patterns [73]. As the authors in [73] say, an analogy between question and answer can be made: quality attributes ask for a solution, and the decisions respond.

Architecture knowledge such as design decisions is created in the analysis, design, and even evaluation of a software architecture [74]. Moreover, the evolutive nature of a

software architecture accepts that new decisions can be made, as well as the obsolete ones removed [75].

Methods such as the Architecture Tradeoff Analysis Method (ATAM) [6] and Decision-Centric Architecture Reviews [32] explicitly emphasize the evaluation of architecture design decisions, although the first takes a risk identification approach, while the latter takes a decision-challenging approach.

A great deal of architecture knowledge is found tacitly [76,77], including assumptions, values, and internalized experiences [78]. For practical use in an architecture evaluation, practitioners must consider the progression of architecture decisions from a tacit identification to a concrete decision record. The problem with tacit knowledge is that it is difficult to communicate and analyze [78], and in the worst cases, it can lead to a phenomenon called "knowledge vaporization" [77,79] (i.e., knowledge is lost forever). The progression from tacit knowledge to explicit knowledge is known as "knowledge conversion" [80]. Particularly important for the progression of Alpha "Architecture Decisions" is the "externalization" mode of knowledge conversion [80]. This mode uses codification [77,78,80] to express knowledge in an explicit form. Codification of tacit knowledge means embracing a dialogue or collective reflection [80] in which stakeholders articulate tacit knowledge to express architecture decisions in a structured way [77]. This structured form typically includes a name for the decision, a rationale explaining "the why" of the decision, the scope, the current state (e.g., decided, rejected, or obsolesced), the author, among other attributes [77]. Some authors also call for including the criteria for choosing one alternative [81].

For example, in an architecture evaluation, the authors observed a tacitly expressed decision about the organization of the modules of the system. After collective reflection in one of the evaluation meetings, this tacit decision progressed to a more articulated representation. This articulation was achieved by collective reflection with several stakeholders. A whiteboard was used to write down the important issues and to characterize the decision in a structured form. This articulated and explicitly defined decision allowed the evaluation to learn that the architecture was already in a desired state: the modules were already organized following a domain-driven approach. Part of this experience is reported in [21].

Once the tacit knowledge that is relevant for the architecture evaluation has been converted to explicit knowledge, the analysis can be approached in a more systematic way. Explicit knowledge is also easier to record for future reference (the authors have been using Architecture Decision Records (ADRs) for this purpose; see Appendix A for a more detailed description of the ADRs used). Decisions expressed as ADRs can be recorded in version control systems. The authors have been using services such as GitHub with semantic versioning (See recommendations for semantic versioning at https://semver.org/, accessed on 8 March 2026) to store ADRs.

The Alpha "Architecture Decisions" comprises the following states (see Figure 7): Tacit identification, Explicit identification, Recorded and addressed.

The States are described as follows:

1. **Tacit identification:** Reaching this state means that architecture decisions are identified at least in a tacit form, that is, no explicit definition or recording is observed. In this state, some knowledge about the rationale behind those decisions is expected (some of them very old as in the case of legacy systems). Some knowledge of who made one or more of these decisions is also expected.
2. **Explicit identification:** At this state, architecture decisions are observed in explicit form. This means that decisions are characterized, although not necessarily recorded, to facilitate clear and concise communication. Some decisions are being (or have been) evaluated according to the evaluation approach. There are observable efforts for recording decisions, although this is still done in an ad hoc manner. In this state, it

is also clear who made the decisions and the rationale behind those decisions. Risks related to design decisions are also being observed as a result of the evaluation and the impact on quality attributes is understood. In this state, relationships between decisions are also expected to be identified.

3. **Recorded and addressed:** In this state, design decisions have been recorded in a specific format for this purpose (for example, the Architecture Decision Record or ADR ("Documenting Architecture Decisions", Michael Nygard, 2011, accessed on 8 March 2026, https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions)). Decisions have been reviewed according to the evaluation approach, and the reporting of such reviews presents risks identification and the impact on quality attributes. Due to that, at this state, it is highly expected that some decisions are added, deprecated, or improved, it is also expected that the use of a specific version control system, ideally supported by a specific-purpose tool.

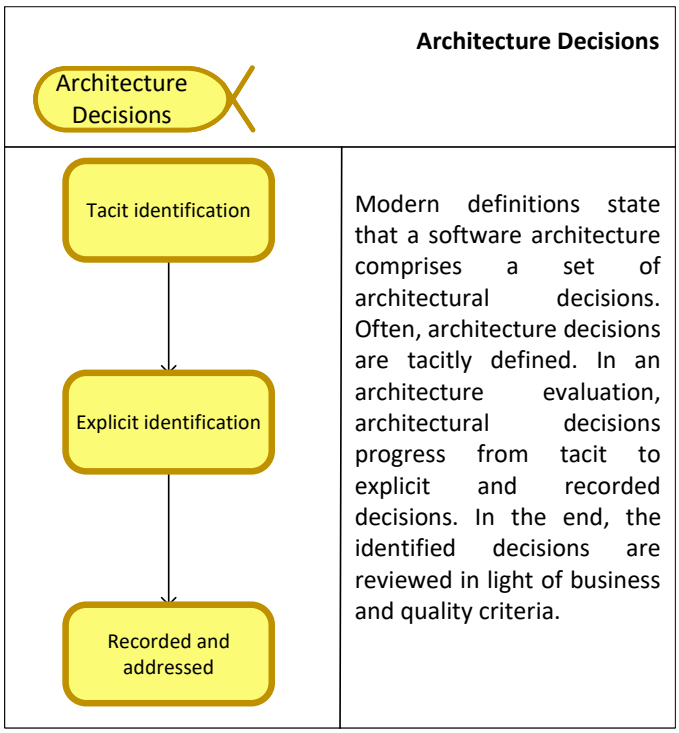


Figure 7. The States of Alpha "Architecture Decisions".

4.6. Alpha: Architecture Description

The Alpha "Architecture Description" represents the elements that are used for expressing, communicating, and analyzing a software architecture. The ISO/IEC/IEEE 42010:2022 "Software, systems and enterprise: Architecture description" [82] presents an architecture description as a work product that expresses an architecture of an entity of interest (e.g., a software product). As the standard goes generic when talking about architectures, the entity of interest is not explicitly specified [82]. Taylor and van der Hoek [14] agree that the availability of artifacts that allow the analysis of a software architecture is one of the intrinsic challenges of architecture design. Such artifacts are even more critical in globally distributed development [83].

Even if some architecture description exists at the time of the evaluation, this Alpha stresses that an architecture description must reflect actual and current architecture. If an architecture description does not express the current architecture, the evaluation team risks ignoring key architectural issues such as technical debt items [84] (the gap between

architecture description and its implementation is a well-known problem in software architecture [85]).

Architecture descriptions are intended to express architecture abstractions, and therefore, they are key for reasoning about the architecture [68,86].

For this work, the conceptualization of a software architecture as a set of architecture-level design decisions was adopted.

The Alpha "Architecture Description" comprises the following states (see Figure 8): General overview, Models developed, Completed.

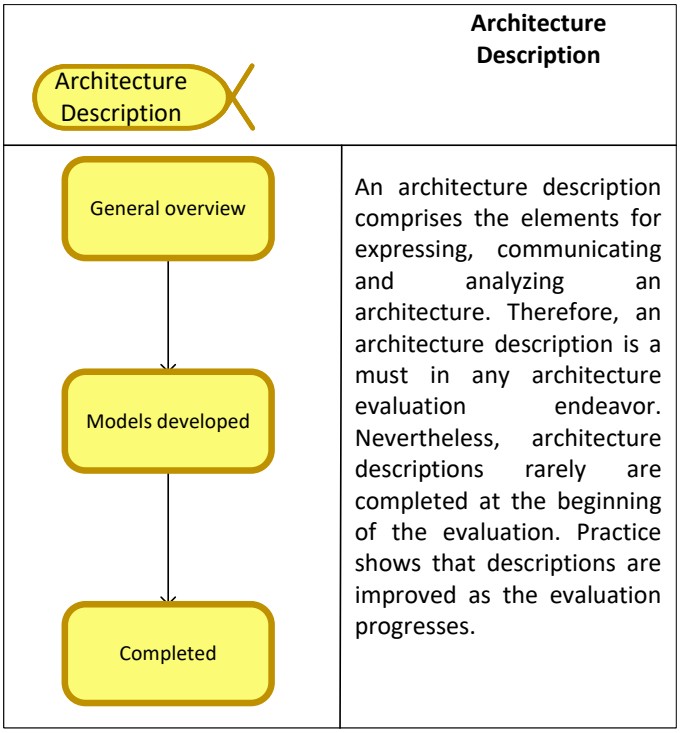


Figure 8. States of Alpha "Architecture Description".

The States are described as follows:

1. **General overview:** At this state, a general overview of the architecture exists. This overview might be composed of one or more diagrams that adapt the abstraction to the stakeholders' concerns. Typically, this overview depicts a high-level view of the architecture components and their deployments. For this state to be reached, a specific configuration of components and connectors is not required.
2. **Models developed:** Reaching this state implies that the architecture description is now encompassing a set of several models presenting more specific system configurations (components-connectors). The description includes views and viewpoints, and they are justified for evaluation purposes and the concerns of stakeholders. In this state, an architecture description is expected to be expressed informally (e.g., diagrams on a whiteboard complimented with natural language).
3. **Completed:** To reach this state, the architecture description is considered complete for the software system whose architecture is under evaluation. At this state, a description in a specific language such as ArchiMate, UML, SysML, and ADL, among other alternatives, is expected. The use of such languages allows the evaluation team to adopt a modeler tool, which in turn enables the adoption of more systematic practices such as configuration management and version control.

4.7. Alpha: Business Goals

The Alpha "Business Goals" represents the intention that an organization has in relation to a software system. Business goals describe where an organization wants to end up [87] and they can express what the organization tries to achieve with the software system [88].

Not all business goals have an effect on a software system and its architecture [89–91]. However, if a business goal has a recognizable effect in the system, it provides the reason for some of the requirements, especially the quality attribute requirements [92], which in turn affect the architecture design [90,91,93].

Understanding business goals at an appropriate level is key for a software architecture evaluation because the software and its architecture should be aligned with these goals. In other words, business goals must be characterized to be suitable for the analysis [89].

The characterization of business goals follows an articulation from more abstract intentions towards more concrete definitions of requirements. The authors have extensively used two approaches to articulate business goals. The first approach comes from the "Pedigreed Architecture eLicitation Method" (PALM) [90], where the main idea is to identify quality attributes that, if attended in the architecture, would help achieve the business goal. The second approach is the "NFR Framework" [94] in which the idea is to iteratively decompose the requirements that initially are specified as *softgoals* into more concrete *softgoals* until reaching the level of *operational softgoals*.

In practice, the first approach allows for the identification of a business goal that, in some way, influences the software system. Then, the business goal can be treated as a high-level softgoal, which is iteratively decomposed into a quality attribute and then to more concrete architectural alternatives that are said to operationalize the goal [94]. For example, in one architecture evaluation, the authors observed a business goal expressed as "increase market share" for the signature software product of the company (a payroll system). In the evaluation, this business goal turned out to be related to a quality attribute of performance. The performance attribute then turned into a principle that guides the architecture to support high performance in calculating payroll in companies with many employees and each employee with more than two liquid assets.

Strategic-level business goal definitions, which are organization-wide, tend to appear very abstract in their nature. The articulation of such goals is also important because it allows one to understand how these goals shape the intention that the organization has with the software system. For example, the authors observed once that a cost reduction business goal was translated to a more concrete software-level goal: the migration from an expensive database engine to a free and open-source alternative.

Methods such as the Architecture Tradeoff Analysis Method (ATAM) [6] explicitly stress the identification and characterization of business goals, while other methods such as the Decision-Centric Architecture Review [32] rely on the implicit assumption that business goals have been characterized.

From the above discussion, it becomes clear that some quality attribute requirements are not directly affected by business goals. For example, the adoption of a domain-driven design can be related to the "modularity" quality attribute, which is hardly traceable to a business goal. In practicing architecture evaluation in many systems, the authors have observed this issue several times, motivating the distinction between Alphas "Quality Attributes" and "Business Goals" in this proposal.

The Alpha "Business Goals" comprises the following states (see Figure 9): Identified, Principles established, Addressed.

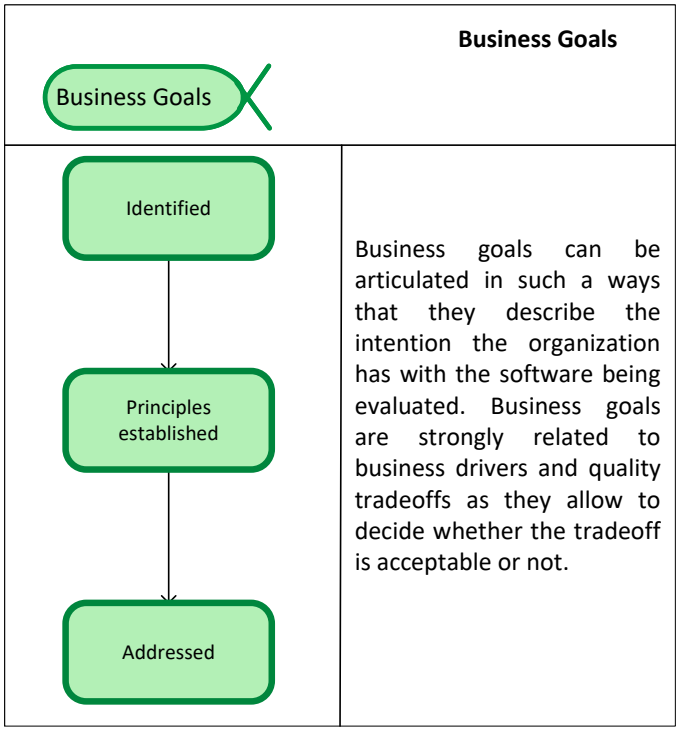


Figure 9. The States of Alpha "Business Goals".

The states are described as follows:

1. **Identified:** At this state, the organization's business goals (or mission goals) are explicitly identified. This means that the business goals are characterized in an appropriate form so that they can be used for the architecture analysis. The business drivers for the architecture are also identified.
2. **Principles established:** At this point, statements about the definition of the software architecture are clear and explicitly identified. Again, this means that these definitions are expressed and characterized appropriately for architecture analysis purposes. The principles that guide the architecture being evaluated are also identified, and they are being used to evaluate the adequacy for the business (or mission) goals.
3. **Addressed:** Reaching this state means that the architecture evaluation has answered how the evaluated software architecture aids in and/or risks meeting the business (or mission) goals that justify the software existence. In addition, a discussion of the tradeoffs between the business (or mission) goals in relation to the reviewed architecture is expected.

4.8. Alpha: Evaluation Adoption

The Alpha "Evaluation Adoption" represents the expected path for a healthy practice adoption. It consists of the progression path for the use of the software architecture evaluation practice. The claimed benefits of a software practice, such as architecture evaluation, will be observed if practitioners can integrate the practice into the development effort [95].

This Alpha progresses from the selection of an appropriate method or approach for the architecture evaluation to the organizational memory accrued. The authors have used their reflective experience in architecture evaluation to propose this path for a more natural adoption of architecture evaluation.

The Alpha "Architecture Evaluation" comprises the following states (see Figure 10): Method or approach chosen, Method or approach integrated, Working well, Organizational memory accrued.

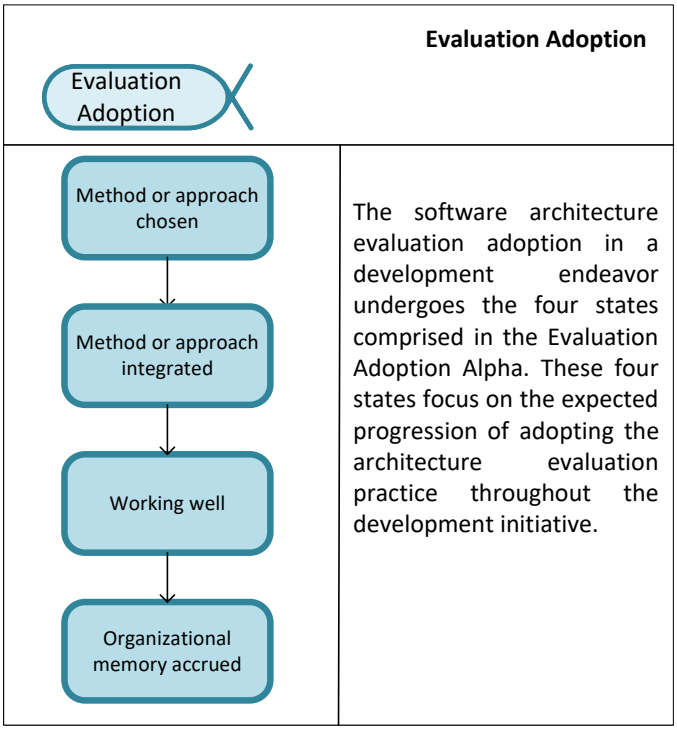


Figure 10. The States of Alpha "Evaluation Adoption".

The States are described as follows:

1. **Method or approach chosen:** Reaching this state means that the evaluation methods or approaches have been reviewed, the criteria for the selection of one method or approach have been agreed upon, and as a consequence, the method or approach has been chosen. The authors have observed that not all stakeholders understand the benefits of running an architecture review, and thus, in this state, it is also expected that stakeholders have agreed that the architecture evaluation is justified.
2. **Method or approach integrated:** At this state, it is expected that the method or approach has been explained to the evaluation stakeholders. Steps, activities, roles, work products, and expected responsibilities and effort are understood and agreed upon. If for any reason one or more stakeholders find that the method or approach should be re-assessed in terms of its suitability for the case, then it is expected that at this point a discussion has already occurred.
3. **Working well:** Reaching this state means that deviations between planned and actual activities have been corrected and the evaluation team and stakeholders are working in a seamless way. The results being delivered (partial or final) are expected to be consistent. The use of sound tools (e.g., software modelers, configuration management software) is expected, and stakeholders agree that the evaluation goals are being met.
4. **Organizational memory accrued:** At this point, the architecture evaluation inherent tasks are finished. Reaching this state means that the organization has accrued in some form the significant knowledge learned from the evaluation experience. The organization might have started accruing evaluation experience knowledge long before reaching this state (for example, in Scrum, it is reasonably that this "knowledge accruement" started as soon as the team starts reflecting about the process in the retrospectives). For this state to be reached, the focus is on architecture evaluation

process-related knowledge, rather than architecture-related knowledge (e.g., architecture decisions, a matter of the Alpha "Architecture Decisions"). Knowledge might be non-articulated (i.e., tacit knowledge) or articulated and recorded. The Alpha in this state does not prescribe any concrete mechanism for knowledge management, as this is organization-dependent. What is important in this state is to understand that process-related memory is an important element to improve organization performance [96].

4.9. Alphas and Relationships

In accordance with the previous definitions for each Alpha, the following relationships are recognized:

- "Business Goals" drive "Quality Attributes."
- "Architecture Description" express "Architecture Decisions."
- "Architecture Decisions" influence "Quality Attributes."
- "Evaluation Adoption" reviews "Architecture Decisions."
- "Evaluation Adoption" improves "Architecture Description."
- "Evaluation Adoption" examines "Business Goals," and "Quality Attributes."

See Figure 11 for Alphas and their relationships using the SEMAT Kernel [24] graphical notation grouped by area of concern.

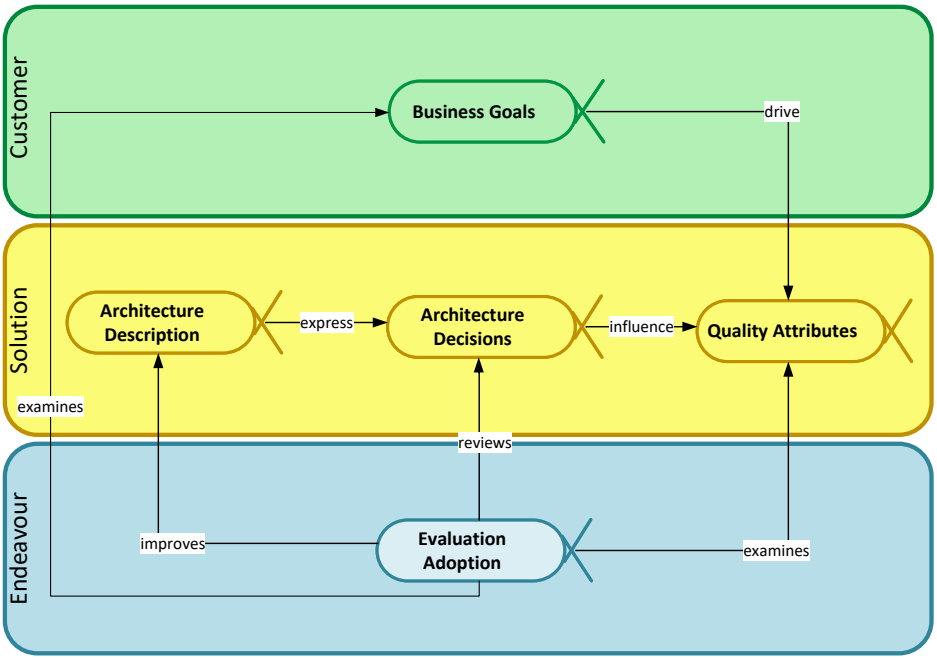


Figure 11. Relationships between the Software Architecture Evaluation Alphas represented using SEMAT's graphical notation.

These relationships express the connection between these "essential" elements that practitioners work with in an architecture evaluation, and therefore, they do not imply any ordering in the use of the Alphas of this proposal.

4.10. A How-To Discussion

In practical use of this proposal, the authors have observed two difficulties that practitioners face when making first-time use of the proposal:

- How Alpha State Cards are used.
- What about Stakeholders (and other "essentials" in software engineering).

These difficulties are discussed in this section to help mitigate a potential "cold start" in the use of the Alphas proposed in this work.

4.10.1. How to Use the Alpha State Cards to Guide and Assess a Software Architecture Evaluation

The reader should recall that each Alpha progresses through a predefined number of States (see Section 4.1). The progression is a consequence of meeting the criteria for each State that is expressed in an Alpha State Card (see Section 4.1).

For example, Figure 12 shows two Alpha State Cards from the proposal. These two Alpha State Cards present the States 1 ("Tacit identification") and 2 ("Explicit identification") for Alpha "Architecture Decisions". In the example, for an engineering team to say that State 1 is achieved, they must find evidence that supports that the checklist items are met. In addition, an engineering team will seek to meet the criteria of State 2 to progress in this Alpha.

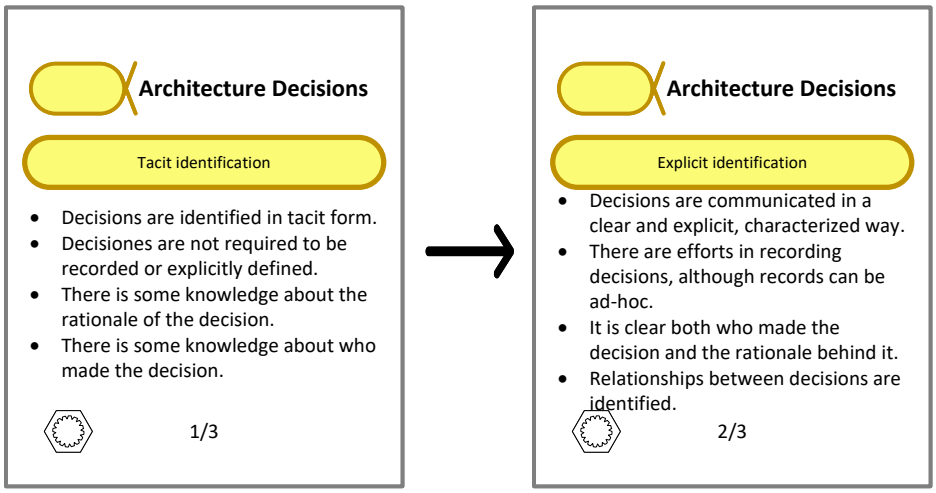


Figure 12. States 1 and 2 from Alpha "Architecture Decisions" represented using SEMAT's graphical notation for state cards (the arrow indicates the progression direction).

In practice, an architecture evaluation is guided and its progression assessed by doing the necessary work to meet the criteria of the current and following States in an incremental way. If after making an assessment with the Alphas, the team finds that no State is still reached, then the evaluation team is required to perform the necessary work to satisfy the checklist items for the first State.

The authors do not prescribe a particular order for the use of Alphas. Indeed, the authors expect these Alphas to be worked with different efforts, that is, following an iterative and incremental manner. The ordering in which the Alphas will be attended depends on the particular product, project, or even organizational context. For example, if an organization has already been working with architecture evaluations and has decided to prescribe a particular method, then the Alpha "Evaluation Adoption" will not be a first priority (especially for the first State).

4.10.2. What About Stakeholders?

Some practitioners have asked "what about stakeholders" in the proposed Alphas. In fact, the proposal does not include an Alpha for stakeholders, and they are critical to a software architecture [4] and its evaluation.

To answer this question, the authors recall that this software architecture evaluation practice representation is also an extension to the SEMAT Kernel [24]. As stated in Section 2, the SEMAT Kernel already includes an Alpha for "Stakeholders" [24]. Thus, when using this

representation to guide and assess the progression of an architecture evaluation, the original SEMAT Kernel Alphas are expected to be used as well. The amount of use of the original Alphas will depend on the context. For example, in the second case presented in this paper, the authors observed a very well-developed identification and characterization of stakeholders. Thus, when the authors arrived, the Alpha "Stakeholders" was already in an appropriate state for architecture evaluation purposes.

The answer to this question has been exemplified with the Alpha "Stakeholders" because this has been explicitly noticed by some practitioners when using the proposed Alphas. Nevertheless, this answer applies to any other existing Alpha in the SEMAT Kernel (i.e., Opportunity, Software System, Team, Way of Working, and Requirements) [24].

5. Case Studies

This section presents two cases in which the proposed Alphas have been used to guide a software architecture evaluation in real-world software products. Each case follows a narrative form to present how this proposal is used in each architecture evaluation. This section also provides a discussion about the key aspects observed to provide contextual evidence that supports the suitability of this proposal to guide such architecture reviews.

In both cases, the evaluation leader took the role of the researcher.

5.1. Case Study Design Aspects

This work embraced the following guidelines for the design of case studies [53,97]:

- **Define the case using a real-world problem:** A key recommendation for the design of case studies is to avoid the use of "toy programs" or "toy examples [53,55,97]. In both case studies, the presented case is defined as the use of the proposed Alphas to guide a real-world software product architecture evaluation.
- **Use more than one data collection approach:** Both case studies relied on:
  - **Observation:** this is a first-degree data collection strategy [97] that implies a careful watching of the progression of the architecture evaluation. In both cases, the emphasis was put in "events". For this work, an "event" is a fact that occurs in the context of the evaluation endeavor that affects a change in an invariant condition of the evaluation. An example of an observable event is a "disagreement between participants in the way the evaluation tasks are run". This event makes a change in an invariant condition, for example, the "scenario brainstorming" changing to "scenario brainstorming stopped". Avoiding losing focus on the research approach is critical, and thus only events that have a potential effect in the evaluation effort are considered. For example, given the previous definition, a "phone call" is also an event. However, unless the "phone call" has an effect of research interest, it will not be considered.
  - **Retrospectives:** Brief retrospectives (15–20 min) were used as needed as the formal instance to discuss the progression of the evaluation, as well as impediments that can hinder the evaluation objectives. The retrospectives followed an unstructured approach [97] to allow the evaluation leader to develop the conversation based on the research interest. Nevertheless, the following issues (or issue questions) were always on the mind of the evaluation leader when running the conversation:
    - Are the evaluation participants understanding the current state in the progression of the architecture evaluation?
    - Are the evaluation participants clear on what has already been done in the architecture evaluation?

* Are the evaluation participants clear on what the next expected activities are going to be done in the evaluation effort?
* Does the evaluation team have an overview of the expected work products in the evaluation effort?

In some retrospectives, interesting questions appeared that arose from the stakeholders. For example, in one of the retrospectives in Case Study 2 (i.e., in the military institution), an interesting question was asked by one of the military leaders: How much of the discovered architecture issues and risks were already known by the software and infrastructure teams? Although such questions are more related to the evaluation itself rather than to the use of the proposed Alphas, the discussion is always illuminating to understand the case.

The retrospectives were also used to discuss the classic "retrospective questions" such as what are the identified impediments to doing the work and what the team plans to do in the next days.

– **Archival data:** A third data collection approach is the use of archival data [97]. Archival data are coincidentally necessary in an architecture evaluation because it allows us to define a starting point for eventual analysis in light of potential new architecture decisions. As noted in [97], the main problem with archival data is that despite being critical for the case, the data are already created and thus there is no control over how the data are generated.

- **The use of triangulation to improve the precision of case study research:** the use of triangulation, that is, broadening the viewpoints for approaching the studied object [97,98] is suggested to increase the precision of empirical case study research. In this work, two kinds of triangulation were used:

  – **Source triangulation:** this kind of triangulation promotes the use of more than one data source [97], where the same insights and issues are collected in different contexts. In this paper, two data sources are presented as Case Study 1 and Case Study 2.
  – **Methodological triangulation:** According to [97], this triangulation focuses on diversifying the data collection methods. In this work, observation, retrospectives, and archival data contributed to diversifying data collection methods.

The **goal** in both cases is to explore the suitability of the proposed Alphas to guide and assess an architecture evaluation endeavor. In both cases, **the case** is defined as the guidance of a software architecture evaluation using the proposed Alphas. In the first case, the context is a military institution where the architecture of a human resources system is the object of the evaluation. In the second case, the context is a public cancer-treatment oriented hospital where a health record system's architecture is the object of the evaluation. Both systems for which their architectures were evaluated exhibit the characteristics of legacy systems: they are key for the organization's business or mission success [99], they are considered aged and complex and they encompass key domain-specific knowledge [100] which is still of significant value [101], they typically run on old platforms and maintenance is expensive [102], with code written following old techniques and in old languages [103], and pose significant challenges for software evolution to meet new functional and non-functional requirements [104].

In the first case, the Architecture Tradeoff Analysis Method (ATAM) was chosen as the evaluation method. This decision relied on the intention of the military institution to perform a formal architecture evaluation, for which a published and well-known method served the purpose. In addition, the ATAM has been reportedly used in other military

institutions. This is an interesting example where the lack of experimental control is observed: the decision to use the ATAM was mostly beyond the control of the authors.

To strengthen the evidence from the use of this proposal, the second case was analyzed. In the second case, no specific evaluation method was used: the case was guided and assessed using this proposal alone.

The authors recall that in both cases an evaluation leader was present, taking the role of the researcher, and it was external to both institutions.

Table 1 summarizes the design aspects discussed in this section.

Table 1. Summary of case study design aspects used in this work.

Design Aspect	Explanation
Goal	• Exploring the suitability of the proposed Alphas to guide and assess a software architecture evaluation endeavor.
Cases	• Case Study 1: using the proposed Alphas to guide the software architecture evaluation of a human-resources system in a military institution. • Case Study 2: using the proposed Alphas to guide the software architecture evaluation of an electronic health record system in a public, cancer treatment-oriented hospital.
Data Gathering Strategies	• Active observation: emphasis on events that change the active condition of the architecture evaluation • Retrospectives: using a conversation-oriented, unstructured approach. • Archival data: review of archival data that is necessary for running the architecture evaluation.
Kinds of Triangulation	• Source triangulation: two different contexts in which an architecture evaluation takes place (Case Study 1 and Case Study 2). • Methodological triangulation: use of active observation, retrospectives, and archival data review.

5.2. Case Study 1: Human Resources Software System in a Military Institution

This case describes the guiding and progression assessment of an architecture evaluation of a human resources software system in a military institution. The case involved a permanent evaluation team with 7 people and an approximate number of 50 stakeholders. Most of the stakeholders were day users of the system. Unlike Case 2, in this case, the evaluation team explicitly committed to the use of the Architecture Tradeoff Analysis Method (ATAM) [6].

Most evaluation methods prescribe actions to be done in a very general way. Often, examples are provided in books (examples for the ATAM method are provided in [6]). The proposal of this paper takes another approach: it provides concrete expected status for all of the essential elements (Alphas) that practitioners need to consider when evaluating an architecture. Each discernible progression status is known as a State, and to achieve it, the evaluators must fulfill the concrete results indicated as checklist items. Given that States express an expected status rather than a general prescription of what to do, they are used to audit and check the progression of an architecture evaluation.

The ATAM method [6] prescribes the following steps:

1. Present the method: the method is presented to the evaluation team.
2. Present the business drivers: the business drivers that explain the opportunity that justifies a software system are presented.
3. Present the architecture: a first architecture description is presented, where the emphasis is on explaining how the current architecture follows business drivers.
4. Identify the architectural approaches: current architectural decisions are identified.

5. Generate the quality attribute tree: the quality attribute tree is an artifact that traces quality attributes to stakeholders' concerns and then to concrete scenarios. The quality attribute tree is prioritized.
6. Analyze architectural approaches: the identified architectural approaches are discussed in light of the quality attribute tree. In this step, architectural decisions are studied to determine if they favor or hinder achieving hig-priority scenarios.
7. Brainstorm and prioritize scenarios: more stakeholders join the architecture evaluation, and scenario elicitation is open to a wider discussion.
8. Analyze architectural approaches: the architectural approaches are again analyzed, now considering the new scenarios elicited.
9. Present results: the execution of the method ends with a presentation of the results of the architecture evaluation.

An interesting aspect of this case is that because the team followed the ATAM method, the order in which the Alphas were worked by the team members was to a great extent predefined. For example, Step 1 in the ATAM method indicates to present the method to all stakeholders. When completed, this step helps the team to reach State 2 of the Alpha "Evaluation Adoption". Still, the proposal is observed to be valuable in the Case: in the same example, the Alphas provide a good mechanism for periodically checking the status and progression of the architecture evaluation. The same happens with steps 2, 3, 4, 5, 6, 7, and 8, which are closely related to the Alphas "Architecture Description", "Quality Attributes", "Business Goals", and "Architecture Decisions", for which the order was mostly dictated by the order of the ATAM steps.

The following aspects can be attributed to the proposal of this work in this case:

- Evaluation method/approach selection: ATAM is an evaluation method that assumes that the method is already chosen. In turn, Alpha "Evaluation Adoption" begins with asking stakeholders to analyze evaluation methods/approaches before choosing one (State 1 in Alpha "Evaluation Adoption"). In this Case, this item of the checklist motivated us to better understand the rationale behind the selection of ATAM.
- Concrete check points for determining the current status of an architecture evaluation endeavor: while ATAM prescribes steps that can be used to globally understand the progression, the Alphas provide, by means of their States, concrete items to check for progression and auditing purposes. The team used these checkpoints for auditing progression.
- Concrete expected results for each State: the checklists are defined in a way that specifies concrete results that are needed before advancing to a next State in the progression of the evaluation. For example, the Alpha "Architecture Description" explicitly asks for specific models when advancing to State 2. The ATAM only requires one to present the architecture as a step to be conducted, leaving the evaluators to their experience to decide on the kind of architecture description to elaborate.

The first Alpha used in this case was "Evaluation Adoption". The main criteria the team used for choosing the ATAM method were that the method has been previously reported in military environments (see, for example, [105]). This decision was agreed and signed off, and the team advanced the Alpha to State "Method or approach chosen".

Before starting with the method, a half-day training was conducted with the aim of explaining the goals of an architecture evaluation and explaining the method to key stakeholders. All members of the evaluation team agreed on the expected effort and the expected tasks and work products. The team then advanced the Alpha "Evaluation Adoption" to the second State: "Method or approach integrated".

The evaluation was "officially" started by articulating and eliciting business drivers (step two in the ATAM). The major concern here was to translate the strategic goals into more product-level-related goals. For example, the goal of "cost reduction", which was explicitly stated by the institution, was translated into more specific goals such as "increasing the automation of many of the use cases that the system could support" (i.e., avoiding the costly effort of human intervention to satisfy use cases that were repetitive in nature and thus subject to a high degree of automation). Other, more concrete goals were related to maintainability aspects (i.e., reducing the time and effort required for future feature additions) and to the dependency of costly old components that still have great importance in the system. The team then moved to State "Identified" of Alpha "Business Goals".

At this point, the ATAM prescribes moving to step three, that is, the architecture presentation. Here, the team started working with Alpha "Architecture Description" where it was found that the institution already had several general overviews for the architecture, although they were diagrams created in general-purpose diagramming software. Thus, in this Alpha, the team started working towards state two: "Models developed". At the same time, the team started working with Alpha "Architecture Decisions" by identifying and understanding the main architecture decisions. The evaluation team also discussed the rationale behind many of these decisions, especially to understand why and how some of the old decisions were made. After meeting the criteria to reach the first State in Alpha "Architecture Decisions", the team sought to meet the criteria for the second State: "Explicit identification".

The next step in the ATAM (i.e., the identification of architectural approaches) moved the evaluation effort to State "Explicit identification" in Alpha "Architecture Decisions". The work carried out in this ATAM step was very technical in nature. The evaluation team worked with software engineers and personnel from the technology infrastructure department. The same work with these stakeholders allowed the team to advance the Alpha "Architecture Description" to the State "Models developed", making strong use of a modeler software.

Following the identification of architectural approaches, the team continued with the following ATAM step: the generation of the utility tree. In this case, the utility tree was generated with the collaboration of the "first line" of stakeholders. In this step, the evaluation team started working with Alpha "Quality Attributes". At first, the team started identifying the quality characteristics. The use of the quality models from ISO 25010:2011 [106] helped the team in identifying quality attributes. Before committing to the use of both quality models (product and quality in use), the evaluation leader presented the quality models, and the team discussed its fitness for the software being evaluated. The quality standard was accepted, and then it was used as a guide. Initially, the team screened the quality characteristics and selected those that appeared to be critical to the system. Then, stakeholders started thinking about concerns and scenarios. At this point, Alpha "Quality Attributes" advanced to State "Identified". The team also advanced to State "Principles established" the Alpha "Business Goals" because the same discussion allowed the team to understand the rules and fundamental assumptions that govern the architecture. The identification of quality attributes, concerns, and scenarios in the generation of the utility tree brought about a better understanding of the business goals (in this case, the team preferred to say "mission goals").

For the next ATAM step, the analysis of architectural approaches, the officer in command of the department responsible for this software system development and maintenance was invited, and its participation was very important for the first part of this ATAM step. The discussion took about one full working day distributed over three calendar days. Much

of the discussion concentrated on the prioritization of scenarios and a better articulation of many of them. The second part of this step was more technical, and the team worked toward better descriptions for the architecture and a better articulation of architecture decisions. The team then started mapping scenarios onto the architecture. At this point, the evaluation effort had met the criteria to advance Alpha "Architecture Description" to State "Completed", Alpha "Quality Attributes" to State "Tradeoffs understood", and Alpha "Evaluation Adoption" to State "Working well". The team also started to share the first partial results. The evaluation leader recommended this to maintain stakeholders engaged in the review.

The scenario brainstorming, which is the next step in the ATAM, was a bit tricky in this case. The evaluation team started to discuss the direction that the evaluation was taking. One of the stakeholders claimed "I don't understand what we are doing, what is the point of doing this". The evaluation leader presented the Alphas and marked the items achieved (in all Alphas) and briefly explained the current position and the next activities to expect with emphasis on the Alpha "Quality Attributes". This helped the team regain attention to the current progression and the next steps expected. The evaluation team also recalled Alpha "Stakeholders" (one of the original in the SEMAT Kernel) to identify the users hierarchy in the institution (and express it in a diagram). Several hierarchies were found, and some of them were very deep. The team quickly found that it would be nearly impossible to directly work with all stakeholders involved. The approach used in this evaluation to deal with the massive stakeholder brainstorming was to invite the people responsible for these hierarchies to explain the scenario brainstorming step and, most importantly, to explain to them how a scenario is articulated as well as several hints to elicit them. The evaluation team also helped them prioritize the scenarios when they returned with the elicited ones. At this point, the evaluation has elicited almost 40 scenarios. An estimated 30% of scenarios were added in the brainstorming scenario step (other scenarios were just confirmation for the first ones elicited in the utility tree generation step). Roughly 20% of the elicited scenarios were deemed as low priority, while quality attributes Interoperability, Modifiability, Performance, and Security comprised the key priority scenarios. Some scenarios were considered to be part of the quality characteristic "Functional correctness" while they might have been classified as "Integrity". Given the experience of the evaluation leader in previous ATAM-based evaluations, the team was instructed not to put much effort into this discussion, as the priority was to maintain the focus of the evaluation effort in the review rather in discussions about which quality characteristic has a better name for classifying some scenarios.

The ATAM calls for a new analysis of architectural approaches with all scenarios elicited up to this point. The new analysis improved the articulation of architecture decisions and scenarios. This allowed the team to address the quality attributes that are operationalized by specific scenarios. The team then moved Alpha "Architecture Decisions" to State "Recorded and Addressed" and Alpha "Quality Attributes" to State "Addressed". The team also advanced Alpha "Business Goals" to State "Addressed" because business (or mission) goals were addressed in the articulation of quality attributes and the mapping of scenarios onto the architecture.

Finally, the evaluation team presented the results. A meeting with the officers in command of the departments involved in the use, development, and maintenance of this system was scheduled, including the ones that joined the evaluation endeavor at the scenario brainstorming step. The team then advanced Alpha "Evaluation Adoption" to State "Organizational memory accrued". The stakeholders involved expressed explicit interest in learning from this experience for future consideration.

Before starting the architecture evaluation, the stakeholders indicated that they were doing ad-hoc architecture evaluations. They reported the lack of a systematic approach to organize both architecture evaluation activities and results, and that the software architect role was distributed among two people. The introduction of the Alphas and their States established a set of actionable progression status checking mechanism, which acted in compliment to ATAM. The evaluation team continuously checked the current status and marked States as achieved (and therefore checklists were fulfilled), and then determined the progression and health of the evaluation by analyzing the States already "burnt" and the deviation from the expected items in the checklists. They also reported that results from the ad-hoc evaluations were not systematically approached. Introducing the Alphas permitted the participants to structure both the presentation and the evaluation report along the Alphas with the following sections: introducing the approach (related Alpha: "Evaluation Adoption"), describing the architecture (related Alpha: "Architecture Description"), describing the business and quality goals (related Alphas: "Business Goals" and "Quality Attributes"), and the approved/revised/rejected architecture decisions and the rationale behind their status (related Alpha: "Architecture Decisions"). In addition, other interesting changes observed are:

- The introduction of the Architecture Decisions Records (ADRs), making the team aware of this technique for recording decisions.
- The introduction of a modeler software.

While both ADRs and the use of modeler software are changes that can be discovered by other means, in this case, the introduction of the Alphas and the explicit statement for progressing towards systematic decisions, recording and architecture descriptions as models made the team aware of these aspects.

In the following, a summary of the main value added by the proposed mechanism to this ATAM-based architecture evaluation is provided. For each step of ATAM, the most related Alpha/State is presented along with the main observed value added to the architecture evaluation. This summary is provided to aid readers in understanding what aspects can be attributed to the use of the proposed mechanism (i.e., the Alphas) in this ATAM-based evaluation. Readers should also note that, in practice, it is not possible to define strict boundaries for the Alphas in terms of the ATAM steps. For example, Alpha "Evaluation Adoption" starts before Step 1 "Present the method" in ATAM; however, in the following discussion, the Alpha "Evaluation Adoption" with State 1 progressing towards State 2 appear related to Step 1 in ATAM.

1. **Present the method:**
    - **Related Alpha/State:** Evaluation Adoption (achieving State 1).
    - **Main Value Added:** Concrete guidance: find arguments for justifying the use of ATAM.
    - **Evidence:** Recorded criteria for the use of ATAM: the main recorded criteria is the availability of reported uses of ATAM in military institutions. The availability of examples of the use of ATAM was also noted as an important aspect to motivate the selection of ATAM.
2. **Present the business drivers:**
    - **Related Alpha/State:** Business Goals (State 1 to State 2).
    - **Main Value Added:** No specific value added: State 1 was achieved by preparing a business goals and architecture drivers presentation, which is already called by ATAM. Although a recommendation was made by the evaluation leader, this recommendation is not attributable to the use of the Alpha.
    - **Evidence:** N/A.

3. **Present the architecture:**
    - **Related Alpha/State:** Architecture Description (State 1).
    - **Main Value Added:** Concrete guidance: leverage (already prepared) deployment diagrams (at service-level), avoiding specific configurations.
    - **Evidence:** Two deployment diagrams (no specific modeling language used), presented in PowerPoint slides.
4. **Identify the architectural approaches:**
    - **Related Alpha/State:** Architecture Decisions (State 1 to State 2) and Architecture Description (State 1 to State 2).
    - **Main Value Added:** Concrete guidance: articulate/model specific configurations and consider appropriate views.
    - **Evidence:** First specific configuration modeled: staff career system modules and database with views for components/connectors and interoperability with other systems.
5. **Generate the quality attribute tree:**
    - **Related Alpha/State:** Quality Attributes (achieving State 1) and Business Goals (State 1 to State 2).
    - **Main Value Added:** Concrete guidance: adopt a quality model to avoid lack of and over-representation of quality attributes. Business Goals advancement is considered an effect of the discussion given by the first State in Alpha Quality Attributes.
    - **Evidence:** ISO 25010:2011 quality model adopted.
6. **Analyze architectural approaches:**
    - **Related Alpha/State:** Evaluation Adoption (State 2 to State 3).
    - **Main Value Added:** Concrete guidance: enforce the delivery of partial results; enforce the use of tools for tracking progress.
    - **Evidence:** Articulation of Quality Attributes into concrete requirements delivered to the officer in command; use of spreadsheets to track the progress of the evaluation.
7. **Brainstorm and prioritize scenarios:**
    - **Related Alpha/State:** Quality Attributes (State 2) and Evaluation Adoption (State 3).
    - **Main Value Added:** Concrete checklists to audit progression and to determine (1) what has been achieved and (2) what is left for the activity (related Alpha: Quality Attributes).
    - **Evidence:** Spreadsheet with four columns: Alpha, State, Checklist, and Done (per checklist item). When activities were carried out to achieve the checklist items, the achieved item was marked as Done.
8. **Analyze architectural approaches:**
    - **Related Alpha/State:** Architecture Decisions (State 2 to State 3).
    - **Main Value Added:** Concrete guidance: recording structured decisions subject to version control.
    - **Evidence:** Git-based repository containing ADRs (two folders: one for approved decisions and another for rejected/deprecated decisions).

9. **Present results:**
    - **Related Alpha/State:** All Alphas and primarily the final States of each Alpha.
    - **Main Value Added:** Reports structured following the argument given by the progression of each Alpha, with special emphasis on the last States of each Alpha (the last States represent the desired status of the results of each Alpha).
    - **Evidence:** Both presentation and written report with the following sections: introducing the approach, describing the architecture, describing the business and quality goals, and the approved, revised, or rejected architecture decisions and the rationale behind their status.

Table 2 provides a summary of the key observed effects in the architecture evaluation of the Case 1.

Table 2. Key observed effects in Case 1.

Observed Effect	• Effect 1: Actionable guidance and progression auditing. • Effect 2: Common ground for reporting. • Effect 3: Regain focus in scenario brainstorming.
Related Alpha	• Effect 1: All Alphas. • Effect 2: All Alphas. • Effect 3: Alpha quality attributes and architecture decisions.
Evidence Observed	• Effect 1: calling stakeholders to analyze and justify the use of the chosen evaluation method. Stakeholders' understanding of concrete expected results for each State in each Alpha, although in this case, the ordering of attention of Alphas was dictated by the method. Marking the States achieved as "done" and using the checklists from each State as checkpoints for auditing the progression and health of the architecture evaluation. • Effect 2: use of the Alphas to define both the presentation and written report structures (i.e., the Alphas supported the argument of these communication artifacts). • Effect 3: during discussion about the relation between a planned activity (scenario brainstorming) and the evaluation goal, the Alphas and their States were reviewed to understand how this activity fits in the evaluation effort; stakeholders regained focus on the architecture evaluations by understanding the relation between this activity and the others to the evaluation goal.
Action	• Effect 1: define activities for fulfillment of checklists of Alphas. Communication to stakeholders of the status of the architecture evaluation with specific references to the States of the Alphas. • Effect 2: elaborate final presentation and report with sections containing the results that appeared in the checklists of each State. • Effect 3: stop the discussion and re-plan the activity for eliciting key requirements and scenarios (State 1 to 2 in Alpha "Quality Attributes"). The discussion occurred at the end of the session and thus the elicitation continued at the next meeting.
Consequence	• Effect 1: a SEMAT-based mechanism (i.e., the Alphas) as a guiding and progression assessment/auditing tool was introduced. • Effect 2: architecture evaluation results presentation and reporting were structured using the Alphas to guide the sections' content. • Effect 3: a SEMAT-based mechanism (i.e., the Alphas) for clarifying the relation between the evaluation activities and the evaluation goal was introduced.

Table 3 presents a summary of the contrast before/after within this case. In the contrast summary, for each key attribute a before and after recounting is provided showing which important changes attributable to the proposal were observed.

Table 3. Within-case contrast for Case 1.

Progress tracking	• Before: Informal, ad-hoc evaluation progress tracking. • After: A spreadsheet was created to track the progression of Alphas in terms of the achievement of the items of the checklists of the States.
Reporting	• Before: Non-structured reports; all reporting made in verbal, tacit form to related officers in command. • After: Alphas provided a common ground for structuring both presentation and written report. Included in this aspect is the fact that a modeler software was adopted for the creation of the architecture models that were not only part of the analysis, but also of the reports.
Decision handling	• Before: Architecture decisions not completely identified and in tacit form. Most of the decision-making knowledge was transferred by word of mouth (especially the historical decisions for which no original decision maker was present). • After: Architecture decisions written down in structured format (ADR) and stored and versioned in a Git-based environment.
Stakeholder participation	• Before: Ad-hoc stakeholder participation (i.e., stakeholder involvement as required). • After: Stakeholders alerted of their eventual participation because the Alphas were presented before the architecture evaluation starts. A stakeholder in particular took the role of checking the progression of the evaluation endeavor.
Organizational memory	• Before: Tacit knowledge about previous informal architecture evaluations; most of the knowledge was transferred by word of mouth. • After: A general-purpose repository was defined and used for storing meeting reports and especially templates generated in the evaluation.

5.3. Case Study 2: Health Record System in a Public Cancer-Treatment Oriented Hospital

This case describes the guiding and assessment of a software architecture evaluation for an electronic health record system that was in production and operation in a public cancer treatment-oriented hospital. The number of people directly involved in the evaluation was approximately 15 people, with an evaluation leader (external and taking the role of the researcher), evaluation supporters (external), information technology infrastructure manager, software developers, innovation leaders, and domain experts as the most important stakeholders observed. The evaluation took approximately 4 full days (8 h each), but the evaluation sessions were not held continuously. Rather, in general, half-days were used for meeting with stakeholders, along with offline work such as preparing architecture descriptions and eliciting architecture decisions.

Approximately half of the meetings were held online using a video-based collaboration platform. Electronic mail was also used to share information about the evaluation itself. When the evaluation sessions were held online, the Alpha State Cards, with their states and checklists, were shared with participants using the "sharing screen" option in the video-based collaboration software.

This architecture evaluation took place in the context of a consulting service. Therefore, the original SEMAT Kernel Alphas were also intensively used. An interesting case was observed when using the Alpha "Stakeholders". When using the Alpha "Stakeholders", two more stakeholders were identified. These two new stakeholders were not initially considered. The new identified stakeholders were considered critical for the system's architecture, and thus they were invited to join the architecture evaluation.

The evaluation effort started by using the Alpha "Evaluation Adoption". The participants were told in general terms what an architecture evaluation is and arguments for possible review approaches were provided. The evaluation team agreed that the approach

to be used is a mix of decision-challenging and scenario-based perspectives. The mixing of these two approaches was the reason the team agreed to choose "an approach" rather than "a method". After formally choosing the approach, the team moved to the State "Method or approach chosen" from Alpha "Evaluation Adoption".

Due to the good availability of the stakeholders, the work with Alpha "Architecture Description" followed Alpha "Evaluation Adoption". The first State for this Alpha is "General overview", which is reached if a high-level architecture description overview shows the main components and their deployments. This description did not exist at the time of this evaluation; therefore, the team started working towards elaborating this artifact. Most of the knowledge about the architecture was available in a tacit way. Some knowledge was also captured in an earlier effort by hospital personnel to model hospital processes, especially those related to health care record management. The Alpha's next State asks for creating models. Thus, after creating more elaborate diagrams on a whiteboard, offline teamwork consisted of quickly translating these diagrams to models using UML in a modeler software.

At this point, it was found useful to review the Alpha "Stakeholders", which is an original Alpha in the SEMAT Kernel. Following the progression path of this Alpha to the third State ("Involved"), the team found that communication with two important stakeholders (both related to biomedical engineering) was overlooked. Involving these two new stakeholders opened up a lot of valuable scenario-related information (at this point, still expressed in an ad-hoc way).

Slightly after starting the progression of the Alpha "Architecture Description", and due to the lack of explicit and recorded knowledge, the team started working with Alpha "Architecture Decisions". This Alpha asks for "Tacit identification" of decisions in order to reach the first state. Most of the architectural knowledge was in this form. Assessing this knowledge was the most intensive part of the evaluation in terms of stakeholders' participation. As soon as the team noticed that the knowledge was in tacit form and at the same time widely agreed by stakeholders, the team decided to move towards working towards the State "Explicit identification" of the Alpha "Architecture Decisions".

The next Alpha in consideration was "Quality Attributes". With this Alpha, the team started by explicitly recognizing the key quality attributes for the systems being evaluated. The quality models for product and system use from the ISO/IEC 25010:2011 "System and software quality models" [106] were used to help the team in the identification of quality attributes. The team mapped several "qualities" that appeared in the many health care related laws, especially those related to the ownership, privacy, integrity and confidentiality of electronic health records. The team then advanced this Alpha to the first State: "Identified".

The Alpha "Business Goals" was worked in parallel with the "Quality Attributes" one. In this case, the team held several meetings with stakeholders dealing with the current strategic concerns of the Hospital. After analyzing all business-related information, the team found that the Hospital has a clear view and mission in relation to one critical aspect: the digital transformation, which included the improvement of current software-supported processes. The team found clear statements that were already documented and also found that people in charge of the view and mission were clear about their thoughts and that the information already documented was also appropriate for the evaluation of the current system's design. Thus, this Alpha was advanced to the second state "Principles established".

Because with the progression of previous Alphas the team also discussed the evaluation implications, the team had also implicitly discussed the evaluation approach and the expected results. The team at this point was also presenting preliminary results from the

evaluation, as required by the next State of the Alpha "Evaluation Adoption". The team then moved this Alpha to the second state: "Method or approach integrated".

The work continued to improve the architecture descriptions and better understand current architecture decisions. The team held two meetings especially devoted to this topic, where stakeholders from development and infrastructure departments joined the evaluation. At this point, the team finally finished the architecture models that started being elaborated in previous meetings, and thus the Alpha "Architecture Decisions" was advanced to the State "Models developed". In addition, the team continued working towards State "Explicit identification" of Alpha "Architecture Decisions".

Due to recommendations from development and infrastructure stakeholders, the team continued to work with Alphas "Business Goals" and "Quality Attributes". The team held other meetings with management hospital's stakeholders' that were directly related to this software architecture, as well as people responsible for health-related technology innovation initiatives. The team continued working with Alpha "Quality Attributes" by identifying main tradeoffs between current identified attributes and in consideration of current architecture decisions. As a result, stakeholders agreed on the understanding of the impact of current and potential decisions on quality characteristics and therefore Alpha "Quality Attributes" advanced to State "Tradeoffs understood"

After continuing to work with Alphas "Quality Attributes" and "Business Goals" the team held two meetings to discuss the fitness of the current architecture decisions for the articulated business goals and quality attributes. The team also started working on recording decisions in ADRs ("Documenting Architecture Decisions", Michael Nygard, 2011, accessed on 8 March 2026, https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions), an approach that one of the team members has used before. Using the ADR format, the team identified decisions that are good, decisions that require more work, and decisions that are deprecated and sometimes replaced by new decisions. In parallel, Alpha "Evaluation Adoption" advanced to State "Working well" as in the brief retrospectives, the team members gained consensus of that the architecture evaluation objectives were being met. The team also started presenting the first preliminary results of the evaluation.

When the team decided that reviewing the architecture decisions was finished, the Alpha "Architecture Decisions" were advanced to State "Recorded and addressed". Given that decisions were reviewed in light of business goals and quality attributes, the Alphas "Business Goals" and "Quality Attributes" were advanced to their respective final States, that is, "Addressed".

At this point, the team continued to improve the current architecture descriptions in order to provide the final report, and thus Alpha "Architecture Description" advanced to its final State: "Completed". The team also started working towards finishing the final report, which consisted of a document summarizing key results from the evaluation. As the team also took notes about the progression of the evaluation in brief retrospectives, the Alpha "Evaluation Adoption" moved to the State "Organizational memory accrued".

At the end of the evaluation, the team has challenged about 10 key architectural decisions related to almost 20 priority scenarios. The key quality attributes were Privacy, Integrity, Confidentiality, and Interoperability. The last was related to scenarios describing how the hospital integrates and interoperates with the central metropolitan health care service.

Before starting the evaluation, in this case the stakeholders indicated the need for an architecture evaluation to determine how far the software system was from several goals, including those of digital transformation. The team reported having no previous experience or intention in formally evaluating the architecture. The role of software architect was very

diluted among developers, and the authors believe that this is the reason for the lack of understanding of how to conduct an architecture evaluation. The introduction and brief review of the Alphas made stakeholders aware of what an architecture evaluation is and what to expect as results. In the evaluation itself, and given that the team had no experience in architecture evaluations, the guidance was step by step following each Alpha and their States. As the team did not have experience in architecture evaluation, no other mechanism for checking progression was in place. The progression and health assessment for the architecture evaluation was introduced by using the Alphas, marking the States already "burnt" while leaving States of Alphas not achieved as unmarked. The Alphas permitted the participants to structure the evaluation report (which before the case was non-existent) along the Alphas with the same sections as in Case 1: introducing the approach (related Alpha: "Evaluation Adoption"), describing the architecture (related Alpha: "Architecture Description"), describing the business and quality goals (related Alphas: "Business Goals" and "Quality Attributes"), and the approved/revised/rejected architecture decisions and the rationale behind their status (related Alpha: "Architecture Decisions"). The introduction of the Alphas, especially the Alpha "Architecture Decisions", made the team aware of the Architecture Decisions Records (ADRs). Unlike Case 1, the team was already using a software for modeling software architectures.

Table 4 provides a summary of the key observed effects of using the Alphas in Case 2.

Table 4. Key observed effects in Case 2.

Observed Effect	• Effect 1: Actionable guidance and progression auditing. • Effect 2: Common ground for reporting.
Related Alpha	• Effect 1: All Alphas. • Effect 2: All Alphas.
Evidence Observed	• Effect 1: stakeholders understanding of what is required to advance in each architecture evaluation Alpha. Marking the States achieved as "done" and using the checklists from each State as checkpoints for auditing the progression and health of the architecture evaluation. This was especially valuable when new stakeholder joined the architecture evaluation, who quickly understood the current position in the evaluation effort and her responsibility in the next activities. • Effect 2: use of the Alphas to define the written report structure (i.e., the Alphas supported the argument of these communication artifacts). In this Case, the team has no experience in reporting architectural aspects of the system.
Action	• Effect 1: define activities to fulfill States' checklists. Communication to stakeholders of the status of the architecture evaluation with specific references to the States of the Alphas. Communicate to the new stakeholder what has been done in the evaluation and what is remaining. • Effect 2: elaborate written report with sections containing the results that appeared in the checklists of each State.
Consequence	• Effect 1: a SEMAT-based mechanism (i.e., the Alphas) as a guiding and progression assessment/auditing tool was introduced. • Effect 2: architecture evaluation results reporting was structured using the Alphas to guide the report sections' content.

Table 5 presents a summary of the contrast before/after within this case. For each key attribute, a before and after description is provided showing the important changes attributable to the proposal that were observed.

Table 5. Within-case contrast for Case 2.

Progress tracking	• Before: No formal or informal architecture evaluations were done before. Only simple technical architecture analysis previously done by one of the developers. The analysis and adaptation of the architecture was done in an as-needed way. The adaptation progression tracking was informal and mostly tacit. • After: A spreadsheet to track progression of Alphas in terms of the achievement of the items of the checklists of the State was created and adopted for the architecture evaluation.
Reporting	• Before: As the previous architecture analyses were done by one person and with technical emphasis, only tacit reports existed, and many of them with knowledge vaporized. • After: The Alphas provided a common ground for structuring the written report of the architecture evaluation. The last States of each Alpha were used to guide the explanation of results.
Decision handling	• Before: Most of the architectural knowledge in tacit form. History of the decisions is not well maintained because the team was composed of new developers. • After: Architecture decisions articulated in ADRs and stored under a Git-based version control system.
Stakeholder participation	• Before: Stakeholders considered only for providing requirements. The only previous architecture analyses were done for small requirements implementations and with technical emphasis only. • After: Alphas presented to the evaluation team, and therefore, the eventual stakeholders were alerted about their potential involvement. Nevertheless, the inclusion of a new stakeholder in the architecture evaluation is not attributable to the mechanism itself, as part of this effect was, as explained in Section 5, observed due to the use of one of the original Alphas in the SEMAT Kernel.
Organizational memory	• Before: Organizational memory available as processes, although many of them were related to the hospital services and not to the development activities. • After: The general-purpose repository was used to record templates, meeting records (including retrospectives), and the draft written report (which was stored as example for eventual future architecture evaluations).

6. Case Studies Discussion and Key Insights

The two cases presented in this work exhibit some commonalities. One thing in common is that in both cases, all Alphas reached their last State when the evaluation effort finished. The researchers observed this to be very natural, that is, no resistance was observed when enforcing the rule that, for a State to be achieved, the complete checklist must be fulfilled. In both cases, it was observed that the teams were eager to increase their knowledge about software engineering and architecture quality practices, and the authors believe that this contributed to following the complete progression path of each of the proposed Alphas. The authors believe that when cost and time constraints arise, a more relaxed rule for achieving a State could eventually be used: for example, is it strictly required to fulfill "organizational memory accruement" (an item of the checklist of State 4 in Alpha "Evaluation Adoption") to consider an architecture evaluation as properly conducted? This is an interesting issue, but more study is required to provide a lightweight version of the proposal.

Another aspect that is shared by both cases is that the two systems are considered as legacy ones, and as such, they are still in use and actively maintained, and supporting key processes in these organizations. In addition, the two cases were observed in development

contexts that can be considered as more classic development styles, although not strict waterfall-like approaches. It was also found that both teams were eager to adopt software practices, even when those practices implied disrupting the development tasks that were mostly oriented to maintenance and support. The stakeholders of both institutions were eager to embrace more agile approaches.

Mostly due to the nature of the organization, in the first case the evaluation meetings were required to be held in person at the institution's facilities. In the second case, it was possible to carry some of the evaluation meetings online (i.e., video conference).

In the first case, this proposal was used as a complement to the well-known Architecture Tradeoff Analysis Method (ATAM), while in the second case, the team made use of the Alphas alone. Due to the "real-world" nature of both cases, decisions such as the selection of the ATAM method were outside the control of this research (as noted in Section 5, the Alpha "Evaluation Adoption" was still used in the beginning of the evaluation to understand the rationale behind the selection of ATAM).

An interesting insight in the first case is that even when a method is present, this proposal was valuable because it provides engineering teams with actionable guidance to determine the current position in the progression of the evaluation and to recognize the future steps the evaluation team needs to focus on. In practice, this means that some of the team members started consulting the Alpha cards and started working beforehand on the necessary elements for the next steps, that is, in a more proactive way. Although the methods also provide guidance expressed in a series of steps, they lack explicit and concrete and practical guidance on how to enact an architecture evaluation (this is one of the drawbacks reported in the literature; see Section 2 for a discussion on this). Typically, practitioners can find some books devoted to evaluation methods in which authors express their recommendations from the practice, but the authors believe that this is not an expeditious way in which an architecture evaluation can be guided (especially in teams interested in running their own architecture reviews). In addition, the authors believe that it is not realistic to suppose that people engaged in an evaluation team would read books about this topic while implementing an architecture review.

The Alphas were also key to gaining attention for the architecture evaluation. This was observed in both cases; however, in the first case, the Alphas proved to be key to regain focus on the architecture evaluation tasks when the team lost the orientation because of agitated discussions regarding the way the stakeholder-massive scenario brainstorming was being held.

The evaluation teams used Alphas (although not with graphical notation) to present an overview of the evaluation practice and what stakeholders could expect. This allowed not only to prepare a better presentation of the evaluation approach, but also stakeholders to use the concrete guidance that Alphas provide to begin planning their own schedules and to think about the future efforts expected.

Unlike other experiences in which the authors have used the SEMAT Kernel with the original Alphas to assess software development efforts (two of these experiences were published [107,108]), in this case the authors were not asked how this proposal aligns with other frameworks such as the CMMI. The authors have been told in the past that if the teams were going to adopt the SEMAT Kernel, a key concern is how much the Kernel is aligned with such frameworks to avoid work duplication (in other words, the main concern from practitioners is that if all the work needed to progress in the Alphas from the SEMAT Kernel is aligned with the work needed for progressing through maturity levels in maturity frameworks). The authors believe that in the case of the architecture evaluation Alphas such concerns were not observed because there are no widely used frameworks

or standards such as the CMMI oriented towards guiding and assessing an architecture evaluation endeavor.

When running these evaluations, the authors also observed that the teams were interested in learning more about architecture evaluation by following the indications of the proposed Alphas. Although this is not specifically designed to guide an architecture evaluation training, the authors observed that it actionably allowed team members to guide their own learning about specific topics in architecture reviews. In the second case, the evaluation leader was surprised to see one of the stakeholders joining the following evaluation meetings with new knowledge about the architecture evaluation effort, even considering that this participant was not coming from the software engineering domain.

A final remark is that this proposal facilitated better organization of architecture evaluation reports. In both cases, the reporting consisted of a presentation and a document. The Alphas provide a concise and concrete architecture evaluation practice structure, and this was used to define a common structure and argument for both reports. At the same time, this common structure makes the reports comparable.

Table 6 presents a summary of the contexts and the main observations for each case.

Table 6. Summary of Case Studies.

Characteristic	Case Study 1	Case Study 2
Type of institution	Military.	Public (i.e., state-run) cancer-treatment oriented hospital.
Main function	Human Resources Management System.	Electronic Health Record System.
Type of system	Legacy, still in use and critical.	Legacy, still in use and critical.
Method or approach	Use of the Alphas and the ATAM (Architecture Tradeoff Analysis Method) for guiding and assessing progression.	Use of the Alphas for guiding and assessing progression with a decision-challenging focus.
Number of stakeholders involved	∼60 people, including a permanent evaluation team of 7 people, with one evaluation leader (external to the institution).	∼15 people, including an evaluation leader and evaluation supporters (both external to the Hospital).
Main observations (specific to one case)	• As a compliment for the method, Alphas provide actionable guidance to determine the progression in the evaluation as well as future expected work to complete the review. • In complex scenario brainstorming, Alphas are allowed to regain focus in the architecture evaluation. • The order in which Alphas were worked by the team appeared to be predefined because of the use of a specific evaluation method.	• Alphas alone provide enough actionable guidance for running the architecture evaluation, as well as an appropriate tool to determine the current progression of the architecture review effort. • Use of the Alphas, especially Alpha "Evaluation Adoption" influenced the early results delivery.
Main observations (in both cases)	• Alphas provide a common ground for organizing final reports (presentation and document). • Guided by the Alphas, stakeholders exhibit interest in autonomous learning about the architecture evaluation topic. • Alphas provide general awareness of the evaluation endeavor, especially in the understanding of the expected outcomes, before the evaluation starts.	

In summary, for both cases, the observations that were made to consider a State as achieved in each Alpha are described:

- Alpha: Evaluation Adoption
  - State 1: Discussion about the evaluation approach done, and agreement on the approach to be used reached.
  - State 2: The approach has already been presented to all participants, and partial results from the evaluation execution are being delivered (especially to high management stakeholders).
  - State 3: Stakeholders agreed that the evaluation met their expectations. Tools for writing down partial results and archiving artifacts are shown to be consistent. When a critical deviation from the planned activities appeared, actions were taken to correct the course of the evaluation.
  - State 4: Retrospectives carried out after each in-person evaluation meeting. Tips for the next architecture evaluation written down.
- Alpha: Quality Attributes
  - State 1: First set of quality attributes elicited. Quality model in use is agreed.
  - State 2: Quality attributes prioritized and articulated into explicit requirements and tradeoffs determined.
  - State 3: Priority quality attributes and their respective requirements analyzed, considering their support by the architecture.
- Alpha: Architecture Description
  - State 1: First architecture overview presented. No details of the components are shown.
  - State 2: Relevant architecture configuration views created and stored in version control system.
  - State 3: Complete description (for evaluation purposes) created. Models created using UML and stored in the version control system. In addition, the models created are being used to document the system being evaluated.
- Alpha: Architecture Decisions
  - State 1: Architecture decisions identified, some rationale identified. Written on the blackboard.
  - State 2: Architecture decisions explicitly identified and written down in version control system.
  - State 3: Architecture Decision Records created for each decision. Also maintained in the version control system.
- Alpha: Business Goals
  - State 1: Business goals and architecture drivers determined and written on the whiteboard.
  - State 2: Business goals articulated into concrete quality goals and used in the evaluation.
  - State 3: Stakeholders agreed that the architecture is supporting the business goals to a high degree.

The reader should note that the observations presented correspond to the cases presented in this article. These observations are not mandatory for other evaluations; they can be used as examples.

In the following, for each of the reported observed effects (see Tables 2 and 4) a discussion of the evidence observed is presented. The discussion is articulated around each observed effect, and then the most involved Alpha(s) and its implication are detailed. Items that were more important for the observation are referenced to the identification given in the Appendix B.

1. **Observed effect: Actionable guidance and progression auditing**
   - Involved Alpha(s): Evaluation Adoption, State 3, especially items EA.S3.1 and EA.S3.2.
   - Evidence Type: Creation and adoption of a simple spreadsheet with four columns (Alpha, State, Checklist Item, and Done).
   - Timing: In both cases, State 3 was achieved in session 4 of the architecture evaluation (this is dependent on the particular architecture evaluation).
2. **Observed effect: Common ground for reporting**
   - Involved Alpha(s): Quality Attributes (especially State 3, item QA.S3.2), Architecture Decisions (especially State 3, item AD.S3.1), Architecture Description (especially State 3, item ADS.S3.2), Business Goals (State 3, item BG.S3.2), and Evaluation Adoption (especially State 1, items EA.S1.1 and EA.S1.2, and State 3, item EA.S3.1).
   - Evidence Type: Written reports and presentation (in Case 1) structured in the following form: introducing the approach (explaining activities done and justification for the evaluation method/approach), describing the architecture (a selection of the models developed and related decisions), describing the business and quality goals (prioritized quality and business goals with tradeoff analysis result and how the architecture supports them), and the approved, revised, or rejected architecture decisions and the rationale behind their status. Each section revolved around the specific items of the involved Alphas.
   - Timing: The written reports and the presentation in Case 1 were iterated from the early stages of the architecture evaluation, as the proposed mechanism instructs to deliver partial results to maintain stakeholders engaged. However, the finalization of the reports was made in the last days of the architecture evaluations. In Case 1, this work can be considered as part of the evaluation (Step 9 in ATAM); in Case 2, this work can be considered outside the evaluation as the Alphas do not explicitly mandate a "final report".
3. **Observed effect: Regain focus in scenario brainstorming (only observed in Case 1)**
   - Involved Alpha(s): Quality Attributes (especially States 2 and 3, items QA.S2.3 and QA.S3.1) and Evaluation Adoption (especially State 2, item EA.S2.2).
   - Evidence Type: The most important artifact is the spreadsheet that was used to check the progression regarding the Alpha "Quality Attributes".
   - Timing: In Case 1, this happened in the scenario brainstorming ATAM step, between sessions 4 and 5 of the architecture evaluation.

7. Threats to Validity

The validity analysis concerns with the truth of the claims derived from the empirical experience [109], including experimental and non-experimental ones (e.g., case studies). A threat to validity is any issue that puts the claims of empirical experience at risk [53,109]. Researchers in both quantitative [109] and qualitative [110] domains agree that if validity is an issue in the empirical experience, two aspects are key: (1) there is no objective truth, therefore, validity analysis is mostly qualitative, and (2) it is neither possible nor practical to rule out every validity threat.

Researchers typically need to take into account threats to the following validity types: conclusion validity, internal validity, construct validity, and external validity [53,109]. Some of these types of validity are adapted for qualitative research [110]. For example, conclusion validity analysis tends to be more quantitative in experimental designs [109]

while in non-experimental designs, such as case studies, researchers take a more qualitative approach [53].

Validity is a property of the knowledge claims rather than the methods [109] and therefore threats should be taken into account in relation to the specific empirical experience [53]. This implies that in the design of an empirical experience, a tradeoff between these validity types is expected, and in most cases, some threats are simply accepted [109].

In the following, each validity type is described in the context of the case studies presented in this paper. The threats that the authors considered important for the cases are also discussed.

7.1. Conclusion Validity (Reliability)

Conclusion validity concerns, with issues that affect the claims that are observed in the empirical experience [109]. In case studies, this type of validity is closely related to "reliability" [53]. Reliability concerns, with the extent to which the results observed in the case depend on the empirical experience and not on the researcher. In other words, the reliability in a case study calls for ensuring that if another researcher replicates the case study, the results are consistent.

In this study, the main concerns with conclusion validity are extraneous factors affecting the case, and overestimation of results.

The first concern is about external events that disrupt the architecture reviews. For example, disruptions in evaluation meetings and disagreements between stakeholders. Such external events can make stakeholders take a pessimistic stance about the evaluation experience. Disruptions in evaluation meetings were not frequent in both cases. However, in the case of the architecture evaluation in the military institution there was a moment in which the disagreement between stakeholders became a challenge because evaluation participants disagreed in the way some evaluation activities were being run. The proposal of this paper was valuable for regaining attention in the evaluation effort.

The second concern is about the risk of overestimating the observations. Overestimation is also called researcher bias [110] and can occur when the researcher values too highly the positive or negative results.

7.2. Internal Validity

Internal validity concerns the degree to which the claims come from the empirical experience. For this work, the degree to which it is true that the use of this proposal serves its purpose: guiding and assessing the progression of an architecture evaluation effort. Threats to this kind of validity are influences that the researcher will frequently never know [53].

The lack of experimental control, which is one of the key reasons for designing a case study instead of other empirical methods [53], makes internal validity one of the most compromised validity types [109].

The most important concerns about internal validity in this work are the selection of subjects, the use of an evaluation method (in addition to this proposal), and the history in the evaluation case (history refers to stakeholders actively or passively learning from the case [109]).

The first concern arises from the fact that both cases occurred in the context of real-world settings and products. Therefore, the selection of subjects was not feasible. Instead, the selection of people was a natural consequence of participation in the engineering of both systems. Fortunately, this threat was self-alleviated as all participants in both case studies were not previously trained in architecture evaluation.

The second concern is a direct consequence of the lack of control in the empirical settings that typically motivate the use of case studies rather than experiments. In the first case presented in this article, the use of a specific evaluation method was a decision strongly influenced by the intention of the military institution to achieve a more formal architecture evaluation. The use of an architecture evaluation method in addition to the proposal of this paper makes the researchers question whether the observations can also reasonably be expected in a setting in which the proposal is used alone. This concern motivated the inclusion of a second case study in which the proposal was used alone.

The third concern is about the motivation to learn about software architecture evaluation while the experience is running. Depending on the organizational setting and the product, an architecture evaluation could take several days or even weeks. Although learning about architecture evaluation is beneficial to stakeholders, it can hinder internal validity. The question that emerges here is whether the observable effects of using this proposal are mostly due to the use of the proposal or if there is a hidden influence of stakeholders progressively learning about architecture evaluation. The approach taken to minimize this validity threat was to keep the "offline" times (i.e., the times between the evaluation activities) reduced as much as possible.

7.3. Construct Validity

Construct validity concerns the consistency between theory and observation [53] in the empirical experience. The threats to this validity type are important to take into account in empirical research to assure that the knowledge claims from the empirical experience are derived from the application of a correctly defined technique [109].

The most important concerns about construct validity in this work are overly optimistic expectations from an architecture evaluation, learning enthusiasm, and the evolution of the proposal.

The first concern is observed when the architecture evaluation practice is presented to stakeholders. The expectations can be overly optimistic. For example, the authors have observed in several instances that stakeholders would like to use an architecture evaluation for the sole purpose of identifying architectural technical debt. This poses a risk in the consistency between the architecture evaluation constructs and the observation and knowledge claims from the empirical experience. The problem arises when accepting that an architecture evaluation can be used to identify architectural technical debt, which is not an intended use for architecture evaluations. Construct validity threats such as this example might arise from technical opinions and intuition, as well as from political and economic sources [109]. A solid foundation based on literature research and a reflection-in-action approach are the most important countermeasures to this concern.

The second concern is that participants are eager to learn about the architecture evaluation practice. This enthusiasm is a known threat to construct validity [53] as it could make these participants prone to bias their understanding of the practice in favor of their own expectations. This concern is alleviated by the use of the proposal of this work, which makes stakeholders focus on a more predefined way of working.

Finally, the third concern emerges from the evolution of the proposal. The authors are also learning from the case studies, and thus the proposal was slightly modified to accommodate improvements. Although this might eventually pose a risk to construct validity (i.e., the construct is evolving), the authors limited the adjustments to improvements in the descriptions of the Alphas. No new Alphas were added between the cases (which would have been considered a big change in the construct represented by the proposal).

In addition, a solid foundation on software architecture evaluation from both literature research and reflective practice, and the use of the SEMAT Kernel as a thinking and

modeling framework are the two strategies that the authors used in this work to alleviate the threats to this validity type. The proposal of this work is in itself a guiding artifact that also alleviates the threats to construct validity, as it deters stakeholders from taking other paths in the evaluation effort.

7.4. External Validity

External validity concerns with the extent to which the claims from the empirical experience can be generalized [53] to variations in people, settings, method or treatment, and outcomes [109]. In case studies, this type of validity is particularly important because it is expected to apply the results to other real-world settings that typically differ from the setting in which the case was studied. Almost, if not all knowledge claims derived from the application of some practice (i.e., the "cause") depend on the setting to some degree [109]. In addition, many of the concerns about external validity motivate further research [109].

The most important concerns about external validity in this work are the interaction of the claims from the study with the people involved in the case, the interaction of the claims with the setting, and the fact that both evaluated products were legacy systems.

The first concern about external validity is the interaction of the claims from the study with the people involved in the case, which is given by the fact that the people involved in both cases did not have much experience in architecture evaluation. The threat in this case is that more experienced people could eventually approach the case as a self-guiding experience rather than following the Alphas and their progression. In both cases, the people involved were inexperienced in architecture evaluation. Thus, this issue was more of a concern before starting the cases rather than in the case execution.

The second concern about external validity emerges from the interaction of the claims from the study with the setting. In both cases, a more classic style of development (i.e., waterfall approach) was observed. The question here is whether the claims from the experience still hold in more agile environments. This concern motivates the authors to continue research in such settings.

The third concern is with respect to the fact that both evaluated software architectures were part of legacy systems and not new developments. This concern raises the question of whether the claims from the experience will be similar in a setting where a new software system is being developed. The authors believe that while a software architecture is an architecture regardless of whether it is a new or old development, some issues could appear because of the nature of the processes involved in a new development (e.g., requirements engineering).

Although both the second and third concerns raise interesting issues, the authors are especially interested in the application of this proposal in a more agile setting as future research (see Section 8).

8. Conclusions

Software architecture evaluation is a key software quality practice, yet not straightforward and many times done in informal, ad-hoc ways. To contribute in facilitating the running of such architecture reviews, this study aims to identify (1) the essential elements that practitioners need to consider when evaluating a software architecture and (2) the discrete states that represent the evolution of each of these elements in an architecture evaluation. This paper presents a SEMAT-Kernal-based representation of the software architecture evaluation practice as a set of Alphas and States that represent the essential elements of the software architecture evaluation practice and the discernible progression stages of each element, respectively. The decision of using the SEMAT Kernel as a modeling framework relies upon the SEMAT Kernel extensibility as well as that it provides a

concrete form to describe practices in terms of "essential elements" that express both the characteristics of each element of the practice and their progression paths defined as a set of concrete States.

In addition to the modeling characteristics that are relevant to this study, the SEMAT Kernel was also chosen because of its actionable approach. One of the authors has extensive experience using the original Alphas from the SEMAT Kernel to assess the progression of various software engineering efforts. Its actionable approach enables development teams to concretely determine the current position in the progression of a software engineering effort and which specific objectives the team needs to achieve to continue progressing the endeavor.

This study focused on the following research questions: (1) what are the essential elements that need to be considered when evaluating a software architecture, and (2) what are the discernible progression states of these essential elements that can be used to assess the progression of the evaluation of software architectures. After several software architecture evaluations in which the authors adopted a reflection-in-action research approach, as well as by reviewing the current software architecture evaluation literature and conducting semi-structured interviews with stakeholders, the following Alphas were determined as the essential elements of the software architecture evaluation practice: Business Goals, Quality Attributes, Architecture Description, Architecture Decisions, and Evaluation Adoption. As indicated in the SEMAT Kernel, an Alpha comprises a set of States, with each State representing a discernible status in the progression of a software practice. Therefore, the States in the Alphas of this proposal represent the progression paths through discernible status in an architecture evaluation endeavor.

The authors conducted two additional architecture evaluations for real-world software products using a case study approach aimed at evaluating in practice the suitability of the proposed Alphas for guiding and assessing the progression of an architecture evaluation. In both cases, the software products are considered legacy and under a more classic, although not exactly waterfall, development style.

It was observed that the proposed mechanism allowed practitioners in such contexts to focus on the key aspects of an architecture evaluation and maintain an actionable guide for the progression of an architecture evaluation. Given that the progression paths are described as expected items, each State in each Alpha can be used as check points for auditing an architecture evaluation. Key observed insights also include: it was possible to run a software architecture evaluation using only the Alphas (without relying on an evaluation method), as well as using the Alphas complementing an evaluation method (which the authors regard as a good indicator that the proposal is consistent with how a well-known evaluation method organize and implement such endeavor); the use of the Alphas helped involved stakeholders to regain focus in the evaluation activities when the work became tangled in a stakeholder-massive ATAM-based review; a more autonomous architecture evaluation was observed because of the actionable guidance that the progression path of each Alpha provides; finally, a more consistent structure was also observed for the evaluation reports (both documents and presentations).

The authors remark that these observations were made in two contexts in which teams are well formed and mature, and the development approach is considered as "classic" (although not exactly a "waterfall" approach), rather than agile. The authors believe "classic" environments differ from agile contexts enough to consider studying this proposal in such environments. At the same time, this belief calls for caution when using this proposal in agile environments, where rapid delivery often interferes with typically slower quality practices such as software architecture evaluation.

Practitioners to whom this proposal has been presented frequently mention that (1) setting and maintaining ordering in the architecture evaluation process, (2) sharing knowledge about how a software architecture is run, (3) avoiding work duplication, and (4) defining concrete tasks, roles, and work products are the aspects where this work is most useful. At the same time, maintaining costs bounded has been cited as the least-facilitated aspect. The authors have not thoroughly studied this last aspect; however, the authors believe that this can be the effect of technical people having a low awareness of cost-related issues probably due to a lack of a commonly accepted relationship between software quality aspects and financial aspects. Although this is a very interesting topic, the authors do not consider it a direct future research direction of this work. Other limitations for the proposal include lack of elasticity for the application of the proposal in other software architecture related practices and a potential unsuitability for guiding small software systems' architectures. In the first case, the authors tried to apply this mechanism to identify technical debt. Although this experience has not yet been published, the authors noted that the proposal is too cumbersome for the sole purpose of identifying architectural debt. In the second case, the authors recognize that even small software systems have an architecture. The proposal might be too heavy for evaluating such architectures: in these cases, a more ad-hoc approach would suffice.

As of this writing, the authors remark four future research directions.

8.1. Better Understand Guidance of Software Architecture Evaluations in Agile Environments

Both cases presented in this paper occurred in development contexts exhibiting a more classic development style, although some efforts to adopt more agile approaches were observed. As of this writing, this proposal has been used in more agile environments and with smaller software systems, but its use has not been reflectively observed, as it was done in the two cases presented in this paper.

In relation to this research direction, the authors would also like to determine if some prescribed or recommended ordering for using the Alphas is possible and if some benefits could be observed by following a specified ordering. In the authors' experience, it has always been very natural to start with the first State of Alpha "Evaluation Adoption". However, the authors believe that in some contexts, this might not be the case. For example, in contexts in which architecture evaluations are prescribed at the organizational level and thus teams do not have much control over deciding which evaluation method or approach is used, the first State of the Alpha "Evaluation Adoption" is considered already reached before the evaluation starts. In a case like this example, the Alpha "Evaluation Adoption" will not be the first priority.

8.2. Define Concrete Essentials for the Architectural Technical Debt Identification

In other minor evaluations in which the authors have used the Alphas from this proposal, it has been observed that using them for the sole purpose of identifying architecture technical debt is too heavy. Nevertheless, the observations also tell that essentials such as the Alphas "Architecture Description" and "Business Goals" are naturally required for architectural technical debt identification, and thus these Alphas are necessary. Key issues arise from the interdependence of the Alphas, and thus the authors believe that they can work on another version of this proposal oriented towards identifying architectural technical debt in existing software systems.

8.3. Identification and Definition of Indicators to Assess How Well an Architecture Evaluation Ends

In most cases where this proposal has been used, the evaluation has concluded when all the Alphas reached the last State. However, in some other minor and less formal architecture evaluations in which the proposal has been used, these reviews can still be

considered as finished even if some Alphas have not reached their last State. In such cases, the evaluation meets the expectations while some Alphas still exhibit States to achieve. The authors believe that this proposal can be complimented with a set of indicators that could help practitioners to assess the maturity of the practice, assuming that less mature cases will show an evaluation finished without necessarily reaching the last State of each Alpha.

8.4. Explore the Suitability of the Proposal for Teaching and Learning About Software Architecture Evaluations

In addition to the use of this proposal for guiding and assessing the progression of architecture evaluations, the authors have also used the proposed Alphas to (1) guide an architecture evaluation-related undergraduate informatics engineering thesis work and to (2) organize the software architecture evaluation teaching material in a software architecture undergraduate course. Although these experiences have not been treated as case studies, preliminary insights show that the proposal is also beneficial for students to gain concrete understanding and advice about the software architecture practice, and for instructors to concretely define a teaching path for the evaluation practice.

Author Contributions: Conceptualization, P.C. and H.A.; methodology, H.A.; software (architecture analysis), P.C. and H.A.; validation, P.C. and H.A.; investigation, P.C. and H.A.; resources, M.S.; data curation, P.C. and H.A.; writing—original draft preparation, P.C., H.A. and M.S.; writing—review and editing, P.C., H.A. and M.S.; visualization, P.C.; supervision, H.A. and M.S.; project administration, H.A. and M.S.; funding acquisition, M.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in the study are included in the article, further inquiries can be directed to the corresponding author.

Acknowledgments: The authors thank the Dirección de Postgrado of the Universidad Técnica Federico Santa María (Chile) for its valuable support in this research.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A

This appendix presents three key elements for replication purposes.

Appendix A.1. The AlphaState Cards

The most important element for the replication of the case studies is the set of Alpha cards. They are provided in printable form as a PDF file on https://github.com/pcruzn/arch-eval-semat (accessed on 8 March 2026). The file includes a card for each State in each Alpha, that is, a total of sixteen Alpha State Cards.

Researchers who want to replicate these case studies can print these cards and use them to guide a software architecture evaluation, starting from the first state in each Alpha and progressing by following the next States. The checklists of a State Card must be verified before claiming that the State has been reached.

Appendix A.2. Architecture Decision Records

Architecture Decision Records (ADRs) are important for the architecture evaluation as well as for the case study. The ADRs are used to describe in a structured way the architecture decisions, and they embody key observable, case-study-related aspects.

The template used in this work characterizes each identified architecture decision with the following structure:

- **Title:** the name of the architecture decision written as a noun phrase.
- **Context:** a brief itemized explanation of the context in which the architecture decision was made. Decision forces can be mentioned here.
- **Decision:** a brief itemized explanation of the architecture decision itself.
- **Status:** an indicator that tells if the architecture decision is implemented, deprecated, or subject to improvement.
- **Consequences:** an itemized explanation of the consequences of the architecture decision.

The ADRs can be stored using markdown language (which is supported and interpreted by GitHub (GitHub: https://docs.github.com/es/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/, accessed on 8 March 2026); see an example on https://github.com/pcruzn/arch-eval-semat (accessed on 8 March 2026) .

Appendix A.3. Architecture Evaluation Report Structure

The report of a software architecture evaluation is a key work product because it condenses all the architecture-related information collected in the evaluation activities.

In the cases presented in this article, each report was about 30 pages long and had the following structure:

- An Introduction or Preamble section, describing the circumstances in which the architecture evaluation takes place, the motivation for evaluating the architecture, the approach or approaches used, and the main findings.
- A Business or Mission Motivation section, describing the business or mission context in which the software product has been conceived. This section also describes the goals the organization has with the software system (for example, if the organization wants to reduce costs from a database system contract).
- A Requirements/Decisions mapping section, presenting the qualities (e.g., from a quality model), their prioritization, and their relation to the scenarios or architecturally significant requirements.
- A Decisions section, presenting the decisions analyzed in relation to the software qualities, including their tradeoffs, the decision status, and the main related risks. In this section, it is key to argue how such requirements are supported or inhibited by the architecture.
- A final section summarizing the findings and presenting the main conclusions of the evaluation.

Appendix B

This appendix presents the complete checklists for each State in each Alpha. The checklists are also available as Alpha State Cards on https://github.com/pcruzn/arch-eval-semat (accessed on 8 March 2026). For a State to be achieved, all the items in the specific checklist must be satisfied. A more relaxed rule could eventually be defined, meaning that a State could be marked as achieved if a "good-enough" part of the checklist is fulfilled. In this paper, the results were observed by following the rule that a checklist must be completely fulfilled to reach a State. More study is necessary to determine if a "good-enough" rule can be defined.

Items in the checklists are identified in the paper for referencing purposes only.

Appendix B.1. Alpha: Quality Attributes

Appendix B.1.1. State: Identified

- QA.S1.1— Quality attributes are explicitly identified.
- QA.S1.2—Some form of quality attributes operationalization is observed (e.g., scenarios).
- QA.S1.3—Quality attributes are recorded.

Appendix B.1.2. State: Tradeoffs Understood

- QA.S2.1—The relative importance or priority of quality attributes is established.
- QA.S2.2—The determined prioritization is used to focus on key attributes.
- QA.S2.3—Tradeoffs between quality attributes are analyzed and understood.

Appendix B.1.3. State: Addressed

- QA.S3.1—Prioritized quality attributes were taken into account in the evaluation.
- QA.S3.2—Tradeoffs recorded and explained.

Appendix B.2. Alpha: Architecture Decisions

Appendix B.2.1. State: Tacit Identification

- AD.S1.1—Decisions are identified in tacit form.
- AD.S1.2—Decisions are not required to be recorded or explicitly defined.
- AD.S1.3—There is some knowledge about the rationale of the decision.
- AD.S1.4—There is some knowledge about who made the decision.

Appendix B.2.2. State: Explicit Identification

- AD.S2.1—Decisions are communicated in a clear and explicit characterized way.
- AD.S2.2—There are efforts to record decisions, although records can be ad-hoc.
- AD.S2.3—It is clear both who made the decision and the rationale behind it.
- AD.S2.4—The relationships between decisions are identified.

Appendix B.2.3. State: Recorded and Addressed

- AD.S3.1—Decisions are recorded in a specific format such as ADRs.
- AD.S3.2—Decisions are subject to version control/configuration management.
- AD.S3.3—Key decisions have been reviewed according to the chosen evaluation approach.

Appendix B.3. Alpha: Architecture Description

Appendix B.3.1. State: General Overview

- ADS.S1.1—An architecture description exists in the form of a general overview (most of the time, this overview shows the component deployment).
- ADS.S1.2—The architecture description does not present any specific architecture component-connector configuration.

Appendix B.3.2. State: Models Developed

- ADS.S2.1—The architecture description presents specific (ad-hoc) configurations for the system.
- ADS.S2.2—The description includes important views, and the viewpoints are justified.
- ADS.S2.3—The description can be expressed by means of informal language (e.g., whiteboard diagrams).

Appendix B.3.3. State: Completed

- ADS.S3.1—The architecture description has reached a state that can be considered complete for the system.
- ADS.S3.2—The description is expected to be expressed using description-oriented languages such as Archi, UML, SysML, ADLs, among other alternatives.

Appendix B.4. Alpha: Business Goals

Appendix B.4.1. State: Identified

- BG.S1.1—The business or mission goals are explicitly identified.
- BG.S1.2—Architecture drivers (both business and mission) are identified.

Appendix B.4.2. State: Principles Established

- BG.S2.1—Statements for the definition of the architecture are clear and explicitly identified.
- BG.S2.2—The principles are used to analyze part or all of the architecture.

Appendix B.4.3. State: Addressed

- BG.S3.1—Business goals are used in the evaluation of the architecture.
- BG.S3.2—The results explain the justification for how the architecture would help meet the goals.

Appendix B.5. Alpha: Evaluation Adoption

Appendix B.5.1. State: Method or Approach Chosen

- EA.S1.1—Evaluation methods have been reviewed.
- EA.S1.2—The criteria for choosing an evaluation method have been signed off.
- EA.S1.3—The selection of an evaluation method has been signed off.

Appendix B.5.2. State: Method or Approach Integrated

- EA.S2.1—The method chosen has been explained to the evaluation stakeholders.
- EA.S2.2—Activities or steps defined by the evaluation method are in execution according to the plan and, if required, tailored activities.
- EA.S2.3—Partial results are being delivered to stakeholders.

Appendix B.5.3. State: Working Well

- EA.S3.1—The planned activities of the evaluation are carried out with controlled deviation from the plan.
- EA.S3.2—Consistent use of tools.
- EA.S3.3—Consistent results from the evaluation initiative are being delivered.
- EA.S3.4—Stakeholders agree that the evaluation goals are being met.

Appendix B.5.4. State: Organizational Memory Accrued

- EA.S4.1—A retrospective analysis has been performed.
- EA.S4.2—Effort, practices, and what worked well have been recorded.
- EA.S4.3—The organizational memory has been updated.

References

1. Taylor, R.N.; Medvidovic, N.; Dashofy, E.M. *Software Architecture: Foundations, Theory and Practice*; Addison-Wesley: Boston, MA, USA, 2007.
2. Bass, L.; Clements, P.; Kazman, R. *Software Architecture in Practice*, 4th ed.; SEI Series in Software Engineering; Addison-Wesley Professional: Boston, MA, USA, 2021.
3. Sievi-Korte, O.; Beecham, S.; Richardson, I. Challenges and recommended practices for software architecting in global software development. *Inf. Softw. Technol.* **2019**, *106*, 234–253. [CrossRef]
4. Rozanski, N.; Woods, E. *Software Systems Architecture: Working with Stakeholders Using Viewpoints and Perspectives*, 2nd ed.; Addison Wesley: Boston, MA, USA, 2011.
5. Bosch, J. *Design and Use of Software Architectures: Adopting and Evolving a Product-Line Approach*; ACM Press: New York, NY, USA; Addison-Wesley Publishing Co.: Boston, MA, USA, 2000.
6. Clements, P.; Kazman, R.; Klein, M. *Evaluating Software Architectures: Methods and Case Studies*; SEI Series in Software Engineering; Addison-Wesley: Boston, MA, USA, 2001.
7. Klotins, E.; Gorschek, T.; Wilson, M. Continuous Software Engineering: Introducing an Industry Readiness Model. *IEEE Softw.* **2023**, *40*, 77–87. [CrossRef]
8. Ciceri, C.; Farley, D.; Ford, N.; Harmel-Law, A.; Keeling, M.; Lilienthal, C.; Rosa, J.; von Zitzewitz, A.; Weiss, R.; Woods, E. *Software Architecture Metrics*; O'Reilly Media: Sebastopol, CA, USA, 2022.
9. Fitzgerald, B.; Stol, K.J. Continuous software engineering: A roadmap and agenda. *J. Syst. Softw.* **2017**, *123*, 176–189. [CrossRef]
10. Soares, R.C.; Capilla, R.; dos Santos, V.; Nakagawa, E.Y. Trends in continuous evaluation of software architectures. *Computing* **2023**, *105*, 1957–1980. [CrossRef]
11. Soares, R.C.; Santos, V.d.; Nakagawa, E.Y. Continuous evaluation of software architectures: An overview of the state of the art. In *SAC '22: Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*; Association for Computing Machinery: New York, NY, USA, 2022; pp. 1425–1431. [CrossRef]
12. Dobrica, L.; Niemela, E. A survey on software architecture analysis methods. *IEEE Trans. Softw. Eng.* **2002**, *28*, 638–653. [CrossRef]
13. Hofmeister, C.; Kruchten, P.; Nord, R.L.; Obbink, H.; Ran, A.; America, P. A general model of software architecture design derived from five industrial approaches. *J. Syst. Softw.* **2007**, *80*, 106–126. [CrossRef]
14. Taylor, R.N.; van der Hoek, A. Software Design and Architecture The once and future focus of software engineering. In *FOSE '07: 2007 Future of Software Engineering*; IEEE Computer Society: Washington, DC, USA, 2007; pp. 226–243. [CrossRef]
15. Cervantes, H.; Kazman, R. *Designing Software Architectures: A Practical Approach*, 2nd ed.; Addison-Wesley Professional: Boston, MA, USA, 2024.
16. Madison, J. Agile Architecture Interactions. *IEEE Softw.* **2010**, *27*, 41–48. [CrossRef]
17. Bellomo, S.; Gorton, I.; Kazman, R. Toward Agile Architecture: Insights from 15 Years of ATAM Data. *IEEE Softw.* **2015**, *32*, 38–45. [CrossRef]
18. Cruz, P.; Astudillo, H.; Hilliard, R.; Collado, M. Assessing Migration of a 20-Year-Old System to a Micro-Service Platform Using ATAM. In *Proceedings of the 2019 IEEE International Conference on Software Architecture Companion (ICSA-C)*; IEEE Computer Society: Washington, DC, USA, 2019; pp. 174–181. [CrossRef]
19. Cruz, P.; Salinas, L.; Astudillo, H. Quick Evaluation of a Software Architecture Using the Decision-Centric Architecture Review Method: An Experience Report. In *Proceedings of the Software Architecture*; Jansen, A., Malavolta, I., Muccini, H., Ozkaya, I., Zimmermann, O., Eds.; ACM Digital Library: Cham, Switzerland, 2020; pp. 281–295.
20. Cruz, P.; Ulloa, G.; Martin, D.S.; Veloz, A. Software Architecture Evaluation of a Machine Learning Enabled System: A Case Study. In Proceedings of the 2023 42nd IEEE International Conference of the Chilean Computer Science Society (SCCC), Concepcion, Chile, 23–26 October 2023; pp. 1–8. [CrossRef]
21. Cruz, P.; Astudillo, H. Positive Side-Effects of Evaluating a Software Architecture. In *Proceedings of the Software Architecture. ECSA 2024 Tracks and Workshops*; Ampatzoglou, A., Pérez, J., Buhnova, B., Lenarduzzi, V.,; Venters, C.C., Zdun, U., Drira, K., Rebelo, L., Di Pompeo, D., Tucci, M., et al., Eds.; ACM Digital Library: Cham, Switzerland, 2024; pp. 167–177.
22. Schon, D. *The Reflective Practitioner. How Professionals Think in Action*; Temple Smith: London, UK, 1983.
23. Jacobson, I.; Ng, P.W.; McMahon, P.; Spence, I.; Lidman, S. The Essence of Software Engineering: The SEMAT Kernel: A thinking framework in the form of an actionable kernel. *Queue* **2012**, *10*, 40–51. [CrossRef]
24. Object Management Group, OMG. *Essence—Kernel and Language for Software Engineering Methods*, version 1.2; Object Management Group (OMG): Milford, MA, USA, 2018.
25. Jacobson, I.; Lawson, H.B.; Ng, P.W.; McMahon, P.E.; Goedicke, M. *The Essentials of Modern Software Engineering: Free the Practices from the Method Prisons!* Association for Computing Machinery and Morgan & Claypool: New York, NY, USA, 2019.
26. Cruz, P.; Astudillo, H.; Zapata-Jaramillo, C.M. Extending the SEMAT Kernel to Represent and Assess Software Architecture Evaluations. In Proceedings of the 2023 XLIX Latin American Computer Conference (CLEI), La Paz, Bolivia, 16–20 October 2023; pp. 1–8. [CrossRef]

27. Ernst, N.A.; Klein, J.; Bartolini, M.; Coles, J.; Rees, N. Architecting complex, long-lived scientific software. *J. Syst. Softw.* **2023**, *204*, 111732. [CrossRef]
28. Kazman, R.; Abowd, G.; Bass, L.; Clements, P. Scenario-based analysis of software architecture. *IEEE Softw.* **1996**, *13*, 47–55. [CrossRef]
29. Cámara, J.; de Lemos, R.; Vieira, M.; Almeida, R.; Ventura, R. Architecture-based resilience evaluation for self-adaptive systems. *Computing* **2013**, *95*, 689–722. [CrossRef]
30. Kazman, R.; Klein, M.; Clements, P. ATAM: Method for Architecture Evaluation. Technical Report CMU/SEI-2000-TR-004. 2000. Available online: https://www.sei.cmu.edu/library/atam-method-for-architecture-evaluation/ (accessed on 25 April 2024).
31. Bengtsson, P.; Bosch, J. Scenario-based software architecture reengineering. In Proceedings of the Fifth International Conference on Software Reuse (Cat. No.98TB100203), Victoria, BC, Canada, 5 June 1998; pp. 308–317. [CrossRef]
32. van Heesch, U.; Eloranta, V.P.; Avgeriou, P.; Koskimies, K.; Harrison, N. Decision-Centric Architecture Reviews. *IEEE Softw.* **2014**, *31*, 69–76. [CrossRef]
33. Harrison, N.; Avgeriou, P. Pattern-Based Architecture Reviews. *IEEE Softw.* **2011**, *28*, 66–71. [CrossRef]
34. Reijonen, V.; Koskinen, J.; Haikala, I. Experiences from Scenario-Based Architecture Evaluations with ATAM. In *Proceedings of the Software Architecture*; Babar, M.A., Gorton, I., Eds.; Springer: Berlin/Heidelberg, Germany, 2010; pp. 214–229.
35. Ford, N.; Richards, M.; Sadalage, P.; Dehghani, Z. *Software Architecture: The Hard Parts*; O'Reilly Media: Sebastopol, CA, USA, 2021.
36. Becker, S.; Trifu, M.; Reussner, R. Towards supporting evolution of service-oriented architectures through quality impact prediction. In Proceedings of the 2008 23rd IEEE/ACM International Conference on Automated Software Engineering—Workshops, L'Aquila, Italy, 15–16 September 2008; pp. 77–81. [CrossRef]
37. Knodel, J.; Naab, M. Software Architecture Evaluation in Practice: Retrospective on More Than 50 Architecture Evaluations in Industry. In Proceedings of the 2014 IEEE/IFIP Conference on Software Architecture, Sydney, NSW, Australia, 7–11 April 2014; pp. 115–124. [CrossRef]
38. Maranzano, J.; Rozsypal, S.; Zimmerman, G.; Warnken, G.; Wirth, P.; Weiss, D. Architecture reviews: Practice and experience. *IEEE Softw.* **2005**, *22*, 34–43. [CrossRef]
39. Kazman, R.; Bass, L. Making architecture reviews work in the real world. *IEEE Softw.* **2002**, *19*, 67–73. [CrossRef]
40. Ferber, S.; Heidl, P.; Lutz, P. Reviewing Product Line Architectures: Experience Report of ATAM in an Automotive Context. In *Proceedings of the Software Product-Family Engineering*; van der Linden, F., Ed.; Springer: Berlin/Heidelberg, Germany, 2002; pp. 364–382.
41. Ali Babar, M.; Bass, L.; Gorton, I. Factors Influencing Industrial Practices of Software Architecture Evaluation: An Empirical Investigation. In *Proceedings of the Software Architectures, Components, and Applications*; Overhage, S., Szyperski, C.A., Reussner, R., Stafford, J.A., Eds.; Springer: Berlin/Heidelberg, Germany, 2007; pp. 90–107.
42. Knodel, J.; Naab, M. *Pragmatic Evaluation of Software Architectures*, 1st ed.; Springer Publishing Company, Incorporated: Berlin/Heidelberg, Germany, 2016.
43. Obbink, H.; Kruchten, P.; Kozaczynski, W.; Postema, H.; Ran, A.; Dominick, L.; Kazman, R.; Hilliard, R.; Tracz, W.; Kahane, E. Software Architecture Review and Assessment (SARA) Report, Version 1.0. 2002. Available online: https://philippe.kruchten.com/wp-content/uploads/2011/09/sarav1.pdf (accessed on 8 March 2026).
44. *ISO/IEC/IEEE 42030:2019*; Software, Systems and Enterprise—Architecture Evaluation Framework. ISO/IEC/IEEE: Geneva, Switzerland, , 2019.
45. *ISO/IEC/IEEE 42020:2019*; Software, Systems and Enterprise—Architecture Processes. ISO/IEC/IEEE: Geneva, Switzerland, 2019.
46. Jacobson, I.; Spence, I.; Johnson, P.; Kajko-Mattsson, M. Re-founding software engineering—SEMAT at the age of three (keynote abstract) In Proceedings of the 2012 27th IEEE/ACM International Conference on Automated Software Engineering, Essen, Germany, 3–7 September 2012; pp. 15–19. [CrossRef]
47. Päivärinta, T.; Smolander, K. Theorizing about software development practices. *Sci. Comput. Program.* **2015**, *101*, 124–135. [CrossRef]
48. Jacobson, I.; Huang, S.; Kajko-Mattsson, M.; Mcmahon, P.; Seymour, E. Semat–Three Year Vision. *Program. Comput. Softw.* **2012**, *38*, 1–12. [CrossRef]
49. Ray, P.; Pal, P. Extending the SEMAT Kernel for the Practice of Designing and Implementing Microservice-Based Applications using Domain Driven Design. In Proceedings of the 2020 IEEE 32nd Conference on Software Engineering Education and Training (CSEE&T), Munich, Germany, 9–12 November 2020; pp. 1–4. [CrossRef]
50. Savić, V.; Varga, E. Extending the SEMAT Kernel with the TDD practice. *IET Softw.* **2018**, *12*, 85–95. [CrossRef]
51. Nørbjerg, J.; Dittrich, Y. The never-ending story–How companies transition to and sustain continuous software engineering practices. *J. Syst. Softw.* **2024**, *213*, 112056. [CrossRef]
52. Stake, R. *Multiple Case Study Analysis*; Guilford Publications: New York, NY, USA, 2013.

53. Wohlin, C.; Runeson, P.; Hst, M.; Ohlsson, M.C.; Regnell, B.; Wessln, A. *Experimentation in Software Engineering*; Springer Publishing Company, Incorporated: Berlin/Heidelberg, Germany, 2012.
54. Abma, T.A.; Stake, R.E. Science of the Particular: An Advocacy of Naturalistic Case Study in Health Research. *Qual. Health Res.* **2014**, *24*, 1150–1161. [CrossRef] [PubMed]
55. Wohlin, C.; Rainer, A. Is it a case study?—A critical analysis and guidance. *J. Syst. Softw.* **2022**, *192*, 111395. [CrossRef]
56. Wallace, D.R.; Zelkowitz, M.V. Experimental Models for Validating Technology. *Computer* **1998**, *31*, 23–31. [CrossRef]
57. Yin, R. *Case Study Research and Applications: Design and Methods*; SAGE Publications: Thousand Oaks, CA, USA, 2017.
58. Robson, C. *Real World Research*; Wiley: Hoboken, NJ, USA, 2024.
59. Silverman, D. *Doing Qualitative Research: A Practical Handbook/David Silverman*, 3rd ed.; SAGE: London, UK, 2010.
60. Shull, F. Sharing Your Story. *IEEE Softw.* **2013**, *30*, 4–7. [CrossRef]
61. CMMI Product Team. *CMMI for Development*, version 1.3; Technical Report CMU/SEI-2010-TR-033; SEI/CMU: Pittsburgh, PA, USA, 2010.
62. Herbsleb, J.; Zubrow, D.; Goldenson, D.; Hayes, W.; Paulk, M. Software quality and the Capability Maturity Model. *Commun. ACM* **1997**, *40*, 30–40. [CrossRef]
63. van Lamsweerde, A. Goal-oriented requirements engineering: A guided tour. In Proceedings of the Fifth IEEE International Symposium on Requirements Engineering, Toronto, ON, Canada, 27–31 August 2001; pp. 249–262. [CrossRef]
64. Harrison, N.B.; Avgeriou, P. Leveraging Architecture Patterns to Satisfy Quality Attributes. In *Proceedings of the Software Architecture*; Oquendo, F., Ed.; Springer: Berlin/Heidelberg, Germany, 2007; pp. 263–270.
65. Gorton, I. *Essential Software Architecture*, 2nd ed.; Springer Publishing Company, Incorporated: Berlin/Heidelberg, Germany, 2011.
66. *ISO/IEC 25010:2023*; Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—Product Quality Model. ISO/IEC: Geneva, Switzerland, 2023.
67. Axelsson, J.; Skoglund, M. Quality assurance in software ecosystems: A systematic literature mapping and research agenda. *J. Syst. Softw.* **2016**, *114*, 69–81. [CrossRef]
68. Bachmann, F.; Bass, L.; Klein, M.; Shelton, C. Designing software architectures to achieve quality attribute requirements. *Softw. IEE Proc.* **2005**, *152*, 153–165. [CrossRef]
69. Dutoit, A.H.; McCall, R.; Mistrik, I.; Paech, B. *Rationale Management in Software Engineering*; Springer: Berlin/Heidelberg, Germany, 2006.
70. Perry, D.E.; Wolf, A.L. Foundations for the study of software architecture. *SIGSOFT Softw. Eng. Notes* **1992**, *17*, 40–52. [CrossRef]
71. Bosch, J. Software Architecture: The Next Step. In *Proceedings of the Software Architecture*; Oquendo, F., Warboys, B.C., Morrison, R., Eds.; Springer: Berlin/Heidelberg, Germany, 2004; pp. 194–199.
72. Che, M.; Perry, D.E.; Yang, G. Evaluating Architectural Design Decision Paradigms in Global Software Development. *Int. J. Softw. Eng. Knowl. Eng.* **2015**, *25*, 1677–1692. [CrossRef]
73. Me, G.; Calero, C.; Lago, P. Architectural Patterns and Quality Attributes Interaction. In Proceedings of the 2016 Qualitative Reasoning about Software Architectures (QRASA), Venice, Italy, 5–8 April 2016; pp. 27–36. [CrossRef]
74. Tang, A.; Avgeriou, P.; Jansen, A.; Capilla, R.; Ali Babar, M. A comparative study of architecture knowledge management tools. *J. Syst. Softw.* **2010**, *83*, 352–370. [CrossRef]
75. Jansen, A.; Bosch, J. Software Architecture as a Set of Architectural Design Decisions. In Proceedings of the 5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05), Pittsburgh, PA, USA, 6–10 November 2005; pp. 109–120. [CrossRef]
76. Capilla, R.; Jansen, A.; Tang, A.; Avgeriou, P.; Babar, M.A. 10 years of software architecture knowledge management: Practice and future. *J. Syst. Softw.* **2016**, *116*, 191–205. [CrossRef]
77. Babar, M.A.; Dingsyr, T.; Lago, P.; van Vliet, H. *Software Architecture Knowledge Management: Theory and Practice*, 1st ed.; Springer Publishing Company, Incorporated: Berlin/Heidelberg, Germany, 2009.
78. Tofan, D. Tacit architectural knowledge. In *ECSA '10: Proceedings of the Fourth European Conference on Software Architecture: Companion Volume;* Association for Computing Machinery: New York, NY, USA, 2010; pp. 9–11. [CrossRef]
79. Borrego, G.; Morán, A.L.; Palacio, R.R.; Vizcaíno, A.; García, F.O. Towards a reduction in architectural knowledge vaporization during agile global software development. *Inf. Softw. Technol.* **2019**, *112*, 68–82. [CrossRef]
80. Nonaka, I.; Takeuchi, H. *The Knowledge-Creating Company*; Oxford University Press: Oxford, UK, 1995.
81. Eilon, S. What Is a Decision? *Manag. Sci.* **1969**, *16*, B172–B189. [CrossRef]
82. *ISO/IEC/IEEE 42010:2022*; Software, Systems and Enterprise—Architecture Description. ISO/IEC/IEEE: Geneva, Switzerland, 2022.
83. Salger, F. Software Architecture Evaluation in Global Software Development Projects. In *Proceedings of the on the Move to Meaningful Internet Systems: OTM 2009 Workshops*; Meersman, R., Herrero, P., Dillon, T., Eds.; Springer: Berlin/Heidelberg, Germany, 2009; pp. 391–400.

84. Verdecchia, R.; Kruchten, P.; Lago, P.; Malavolta, I. Building and evaluating a theory of architectural technical debt in software-intensive systems. *J. Syst. Softw.* **2021**, *176*, 110925. [CrossRef]
85. Woods, E.; Rozanski, N. Unifying software architecture with its implementation. In *ECSA '10: Proceedings of the Fourth European Conference on Software Architecture: Companion Volume;* Association for Computing Machinery: New York, NY, USA; pp. 55–58. [CrossRef]
86. Fairbanks, G. Software Architecture is a Set of Abstractions. *IEEE Softw.* **2023**, *40*, 110–113. [CrossRef]
87. Han, E. Setting Business Goals & Objectives: 4 Considerations. 2023. Available online: https://online.hbs.edu/blog/post/business-goals-and-objectives (accessed on 20 March 2025).
88. Kazman, R.; Bass, L. Categorizing Business Goals for Software Architectures. Technical Report CMU/SEI-2005-TR-021. 2005. Available online: https://www.sei.cmu.edu/library/categorizing-business-goals-for-software-architectures/ (accessed on 31 July 2024).
89. Bass, L.; Clements, P., Business Goals and Architecture. In *Relating Software Requirements and Architectures*; Avgeriou, P., Grundy, J., Hall, J.G., Lago, P., Mistrík, I., Eds.; Springer: Berlin/Heidelberg, Germany, 2011; pp. 183–195. [CrossRef]
90. Clements, P.; Bass, L. Business goals as architectural knowledge. In *SHARK '10: Proceedings of the 2010 ICSE Workshop on Sharing and Reusing Architectural Knowledge*; Association for Computing Machinery: New York, NY, USA, 2010; pp. 9–12. [CrossRef]
91. Bass, L.; Clements, P. The Business Goals Viewpoint. *IEEE Softw.* **2010**, *27*, 38–45. [CrossRef]
92. Gross, D.; Yu, E. Evolving system architecture to meet changing business goals: An agent and goal-oriented approach. In Proceedings of the Proceedings Fifth IEEE International Symposium on Requirements Engineering, Toronto, ON, Canada, 27–31 August 2001; pp. 316–317. [CrossRef]
93. Clements, P.; Bass, L. Relating Business Goals to Architecturally Significant Requirements for Software Systems. Technical Report CMU/SEI-2010-TN-018. 2010. Available online: https://www.sei.cmu.edu/library/relating-business-goals-to-architecturally-significant-requirements-for-software-systems/ (accessed on 31 July 2024).
94. Chung, L.; Nixon, B.A.; Yu, E.; Mylopoulos, J. The NFR Framework in Action. In *Non-Functional Requirements in Software Engineering*; Springer: Boston, MA, USA, 2000; pp. 15–45. [CrossRef]
95. Sletholt, M.T.; Hannay, J.E.; Pfahl, D.; Langtangen, H.P. What Do We Know about Scientific Software Development's Agile Practices? *Comput. Sci. Eng.* **2012**, *14*, 24–37. [CrossRef]
96. Morey, D.; Maybury, M.T.; Thuraisingham, B. *Knowledge Management: Classic and Contemporary Works*; The MIT Press: Cambridge, MA, USA, 2000. [CrossRef]
97. Runeson, P.; Höst, M. Guidelines for conducting and reporting case study research in software engineering. *Empir. Softw. Eng.* **2009**, *14*, 131–164. [CrossRef]
98. Stake, R.E. *The Art of Case Study Research*; Sage Publications, Inc.: Thousand Oaks, CA, USA, 1995; p. 175.
99. Hussain, S.M.; Bhatti, S.N.; Ur Rasool, M.F. Legacy system and ways of its evolution. In Proceedings of the 2017 International Conference on Communication Technologies (ComTech), Rawalpindi, Pakistan, 19–21 April 2017; pp. 56–59. [CrossRef]
100. Wagner, C. *Model-Driven Software Migration: A Methodology Reengineering, Recovery and Modernization of Legacy Systems*; Springer: Berlin/Heidelberg, Germany, 2014.
101. Abdellatif, M.; Shatnawi, A.; Mili, H.; Moha, N.; Boussaidi, G.E.; Hecht, G.; Privat, J.; Guéhéneuc, Y.G. A taxonomy of service identification approaches for legacy software systems modernization. *J. Syst. Softw.* **2021**, *173*, 110868. [CrossRef]
102. Bisbal, J.; Lawless, D.; Wu, B.; Grimson, J. Legacy information systems: Issues and directions. *IEEE Softw.* **1999**, *16*, 103–111. [CrossRef]
103. Bennett, K. Legacy systems: Coping with success. *IEEE Softw.* **1995**, *12*, 19–23. [CrossRef]
104. Yarza, I.; Azkarate-askatsua, M.; Onaindia, P.; Grüttner, K.; Ittershagen, P.; Nebel, W. Legacy software migration based on timing contract aware real-time execution environments. *J. Syst. Softw.* **2021**, *172*, 110849. [CrossRef]
105. Barbacci, M.; Wood, W. Architecture Tradeoff Analyses of C4ISR Products. Technical Report CMU/SEI-99-TR-014. 1999. Available online: https://www.sei.cmu.edu/documents/1207/1999_005_001_16760.pdf (accessed on 2 August 2024).
106. *ISO/IEC 25010:2011*; Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—Product Quality Model. ISO/IEC: Geneva, Switzerland, 2011.
107. Cruz, P.; Astudillo, H. Assessing development teams and organizations with the SEMAT Kernel: Lessons learned from a real-world experience. In Proceedings of the 2017 36th International Conference of the Chilean Computer Science Society (SCCC), Arica, Chile, 16–20 October 2017; pp. 1–4. [CrossRef]
108. Cruz, P.; Astudillo, H. Using the SEMAT Kernel for Software Process Assessment and Practices Implementation in an Software Process Improvement Initiative. In Proceedings of the 2018 37th International Conference of the Chilean Computer Science Society (SCCC), Santiago, Chile, 5–9 November 2018; pp. 1–7. [CrossRef]

109. Shadish, W.; Cook, T.; Campbell, D. *Experimental and Quasi-Experimental Designs for Generalized Causal Inference*; Houghton Mifflin: Boston, MA, USA, 2002.
110. Maxwell, J.A. *Qualitative Research Design: An Interactive Approach;* Sage Publications, Inc.: Thousand Oaks, CA, USA, 1996.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.